



IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Estimation, Prediction of Regression Model Residual Analysis: Validating Model Assumptions - I

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Point Estimation
- Interval Estimation
- Confidence Interval for the Mean Value of y
- Prediction Interval for an Individual Value of y

Problem

- Data were collected from a sample of 10 Ice cream vendors located near college campuses.
- For the i^{th} observation or restaurant in the sample, x_i is the size of the student population (in thousands) and y_i is the quarterly sales (in thousands of dollars).
- The values of x_i and y_i for the 10 restaurants in the sample are summarized in Table

Data

Restaurant	Student Population (1000)	Sales (1000)
1	2	58
2	6	105
3	8	88
4	8	118
5	12	117
6	16	137
7	20	157
8	20	169
9	22	149
10	26	202

Python code for scatter plot

```
In [4]: import pandas as pd
import matplotlib as mpl
import statsmodels.formula.api as sm
from sklearn.linear_model import LinearRegression
from scipy import stats
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

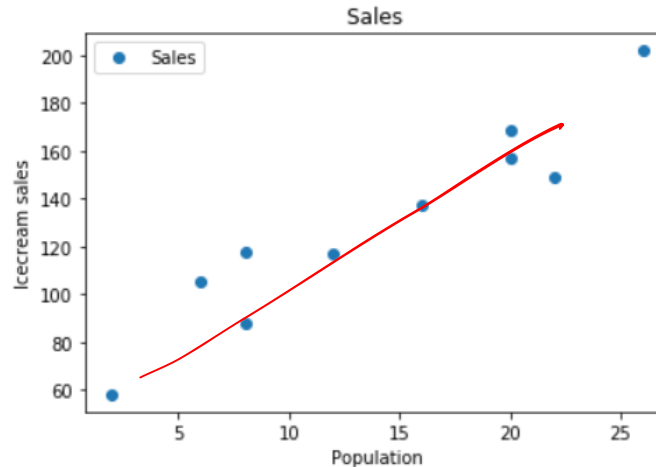
```
In [5]: data = pd.read_excel('C:/Users/Somi/Documents/lrm.xlsx')
data
```

Out[5]:

	Restaurant	Student Population	Sales
0	1	2	58
1	2	6	105
2	3	8	88
3	4	8	118
4	5	12	117
5	6	16	137
6	7	20	157
7	8	20	169
8	9	22	149
9	10	26	202

Python code for scatter plot

```
In [36]: data.plot('Population', 'Sales', style='o')  
plt.ylabel('Icecream sales')  
plt.title('Sales ')  
plt.show()
```



Python code for regression Equation

```
In [37]: import statsmodels.api as sm
St_pop = data['Population']
sales = data['Sales']
st_pop = sm.add_constant(St_pop)
model1 = sm.OLS(sales,st_pop)
result1 = model1.fit()
print(result1.summary())
```

OLS Regression Results

Dep. Variable:	Sales	R-squared:	0.903
Model:	OLS	Adj. R-squared:	0.891
Method:	Least Squares	F-statistic:	74.25
Date:	Wed, 04 Sep 2019	Prob (F-statistic):	2.55e-05
Time:	14:33:11	Log-Likelihood:	-39.342
No. Observations:	10	AIC:	82.68
Df Residuals:	8	BIC:	83.29
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	60.0000	9.226	6.503	0.000	38.725	81.275
Population	5.0000	0.580	8.617	0.000	3.662	6.338

Omnibus:	0.928	Durbin-Watson:	3.224
Prob(Omnibus):	0.629	Jarque-Bera (JB):	0.616
Skew:	-0.060	Prob(JB):	0.735
Kurtosis:	1.790	Cond. No.	33.6

$$\text{Sales} = 60 + 5 \text{ st_pop}$$

Python code for regression Equation

```
In [10]: from sklearn.linear_model import LinearRegression
```

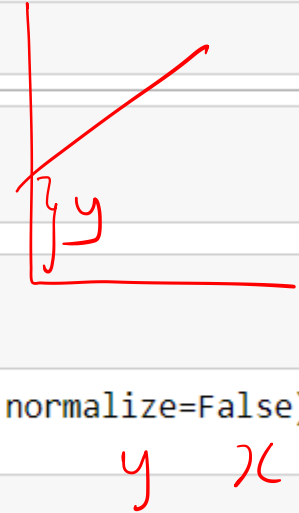
```
In [12]: x = data['Population'].values.reshape(-1,1)
y = data['Sales'].values.reshape(-1,1)
```

```
In [13]: reg= LinearRegression()
reg.fit(x,y)
```

```
Out[13]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1, normalize=False)
```

```
In [14]: reg.intercept_[0], reg.coef_[0][0]
```

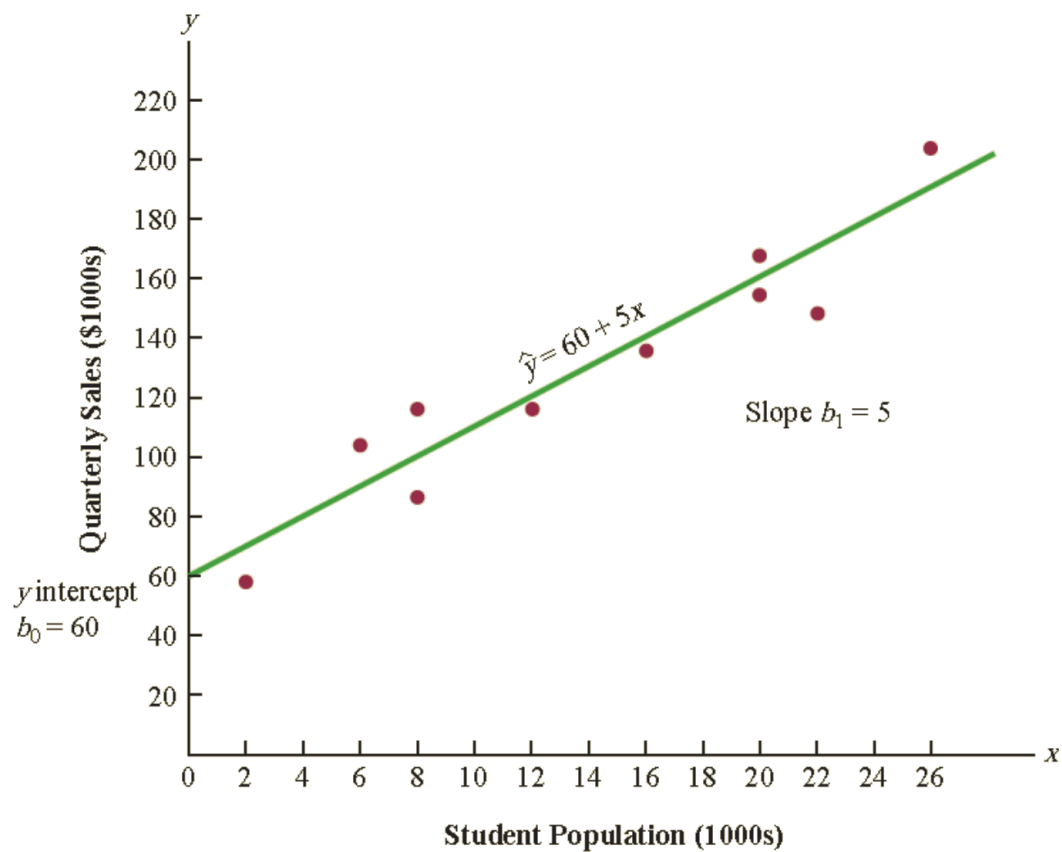
```
Out[14]: (60.0, 5.0)
```



Python code for regression

- In the Ice cream vendor example, the estimated regression equation $60 + 5x$ provides an estimate of the relationship between the size of the student population x and quarterly sales y .

$$\hat{y} = 60 + 5x$$



Point Estimate

- We can use the estimated regression equation to develop a **point estimate of the mean value of y** for a particular value of x or **to predict an individual value of y** corresponding to a given value of x .
- For instance, suppose a manager want a point estimate of the mean quarterly sales for all restaurants located near college campuses with 10,000 students.

Point estimate

- Using the estimated regression equation $60 + 5x$, we see that for $x = 10$ (or 10,000 students), $60 + 5(10) = 110$.
- Thus, a point estimate of the mean quarterly sales for all restaurants located near campuses with 10,000 students is \$110,000.

```
In [32]: reg.predict(10)
```

```
Out[32]: array([[110.]])
```

Point estimate

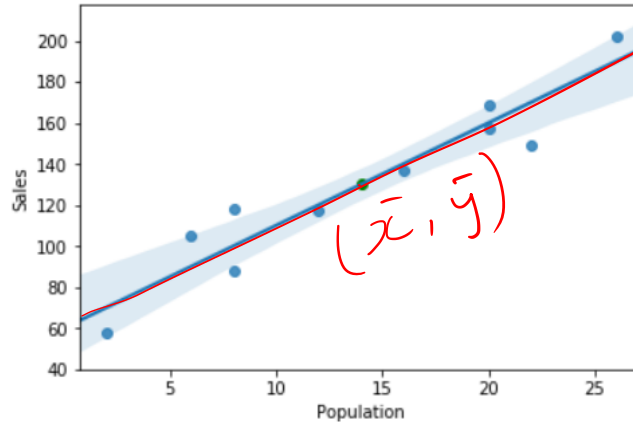
$$\bar{x} \pm 2\sigma$$

- Now suppose the manager want to predict sales for **an individual restaurant** located near College, with 10,000 students.
- In this case we **are not interested in the mean value for all restaurants** located near campuses with 10,000 students;
- We are just interested in predicting quarterly sales for **one individual restaurant**.
- As it turns out, the point estimate for an **individual value** of y is the same as the point estimate for the mean value of y .
- Hence, we would predict quarterly sales of $\underline{60} + 5(10) = \underline{110}$ or \$110,000 for this one restaurant.

Plot at mean value of x and y

```
In [16]: x = data['Population']  
y = data['Sales']  
plt.figure()  
sns.regplot(x,y,fit_reg= True)  
plt.scatter(np.mean(x), np.mean(y), color = "green")
```

Out[16]: <matplotlib.collections.PathCollection at 0x28356eb64a8>



Confidence Interval Estimation

$$E(y) = a + bx$$

- **Confidence interval**, is an interval estimate of the mean value of y for a given value of x .
- **Prediction interval**, is used whenever we want an interval estimate of an individual value of y for a given value of x .
- The point estimate of the **mean value of y** is the same as the point estimate of an **individual value** of y .
- The margin of error is larger for a prediction interval.

Confidence Interval Estimation

x_p = the particular or given value of the independent variable x

y_p = the value of the dependent variable y corresponding to the given x_p

$E(y_p)$ = the mean or expected value of the dependent variable ' y ' corresponding to the given x_p

$\hat{y} = b_0 + b_1 x_p$ = the point estimate of $E(y_p)$ when $x = x_p$

$$60 + 5(10) = 110.$$

Confidence Interval Estimation

In general, we cannot expect $\hat{y}_p$ to equal $E(y_p)$ exactly.

If we want to make an inference about how close $\hat{y}_p$ is to the true mean value $E(y_p)$, we will have to estimate the variance of $\hat{y}_p$.

The formula for estimating the variance of $\hat{y}_p$ given x_p , denoted by $s_{\hat{y}_p}^2$, is

$$s_{\hat{y}_p}^2 = s^2 \left[\frac{1}{n} + \frac{(x_p - \bar{x})^2}{\sum (x_i - \bar{x})^2} \right]$$

Confidence Interval Estimation

CONFIDENCE INTERVAL FOR $E(y_p)$

$$\hat{y}_p \pm t_{\alpha/2} s_{\hat{y}_p}$$

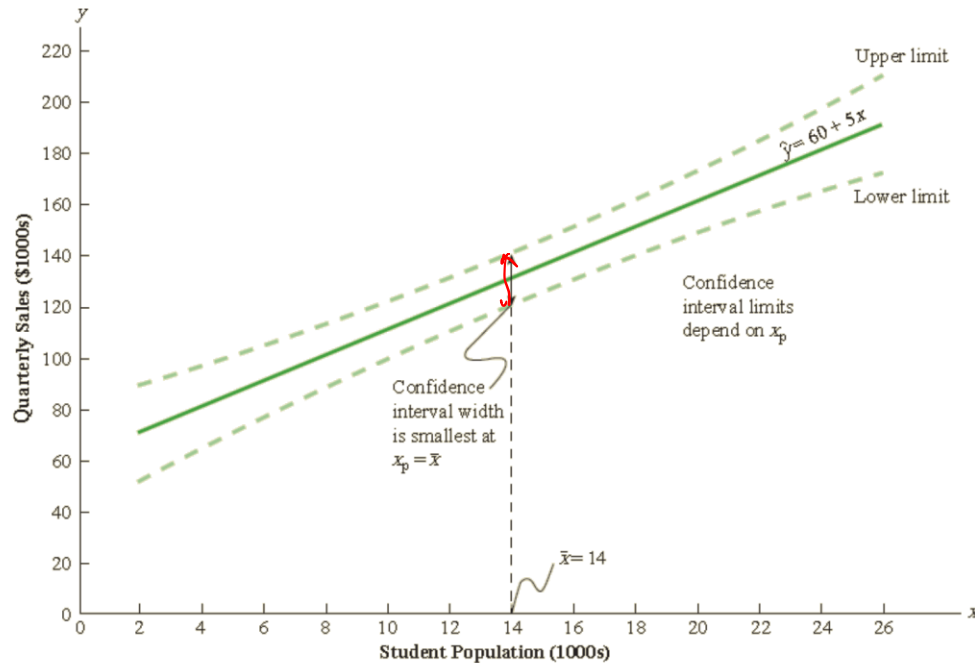
$$\bar{X} \pm z \frac{\sigma}{\sqrt{n}}$$

$$s_{\hat{y}_p}^2 = s^2 \left[\frac{1}{n} + \frac{(x_p - \bar{x})^2}{\sum (x_i - \bar{x})^2} \right]$$

$$\begin{aligned} s_{\hat{y}_p} &= 13.829 \sqrt{\frac{1}{10} + \frac{(10 - 14)^2}{568}} \\ &= 13.829 \sqrt{.1282} = 4.95 \end{aligned}$$

$$\underline{110 \pm 11.415}$$

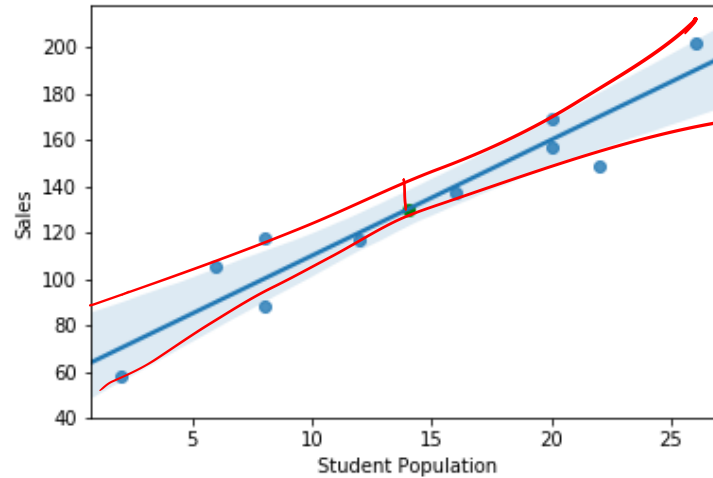
Confidence Intervals for the Mean sales y at given values of student population x



Python Code

```
In [39]: x = data['Student Population']  
y = data['Sales']  
plt.figure()  
sns.regplot(x,y,fit_reg=True)  
plt.scatter(np.mean(x), np.mean(y), color = "green")
```

Out[39]: <matplotlib.collections.PathCollection at 0x1fe5e250a90>



Special Case

The estimated standard deviation of $\hat{y}_p$ is smallest when $x_p = \bar{x}$ and the quantity $x_p - \bar{x} = 0$

$$s_{\hat{y}_p} = s \sqrt{\frac{1}{n} + \frac{(\bar{x} - \bar{x})^2}{\sum (x_i - \bar{x})^2}} = s \sqrt{\frac{1}{n}}$$

Prediction Interval for an Individual Value of y

- Instead of estimating the mean value of sales for all restaurants located near campuses with 10,000 students, we want to estimate the sales for an individual restaurant located near a particular College with 10,000 students.
- (1) The variance of individual ' y ' values about the mean $E(y_p)$, an estimate of which is given by s^2
 - (2) The variance associated with using $\hat{y}_p$ estimate $E(y_p)$, an estimate of which is given by $s_{\hat{y}_p}^2$

Prediction Interval for an Individual Value of y

$$\begin{aligned} s_{\text{ind}}^2 &= s^2 + s_{j_p}^2 \\ &= s^2 + s^2 \left[\frac{1}{n} + \frac{(x_p - \bar{x})^2}{\sum (x_i - \bar{x})^2} \right] \\ &= s^2 \left[1 + \frac{1}{n} + \frac{(x_p - \bar{x})^2}{\sum (x_i - \bar{x})^2} \right] \quad \checkmark \end{aligned}$$

Prediction Interval for an Individual Value of y

$$\begin{aligned}s_{\text{ind}} &= 13.829 \sqrt{1 + \frac{1}{10} + \frac{(10 - 14)^2}{568}} \\&= 13.829 \sqrt{1.1282} \\&= 14.69\end{aligned}$$

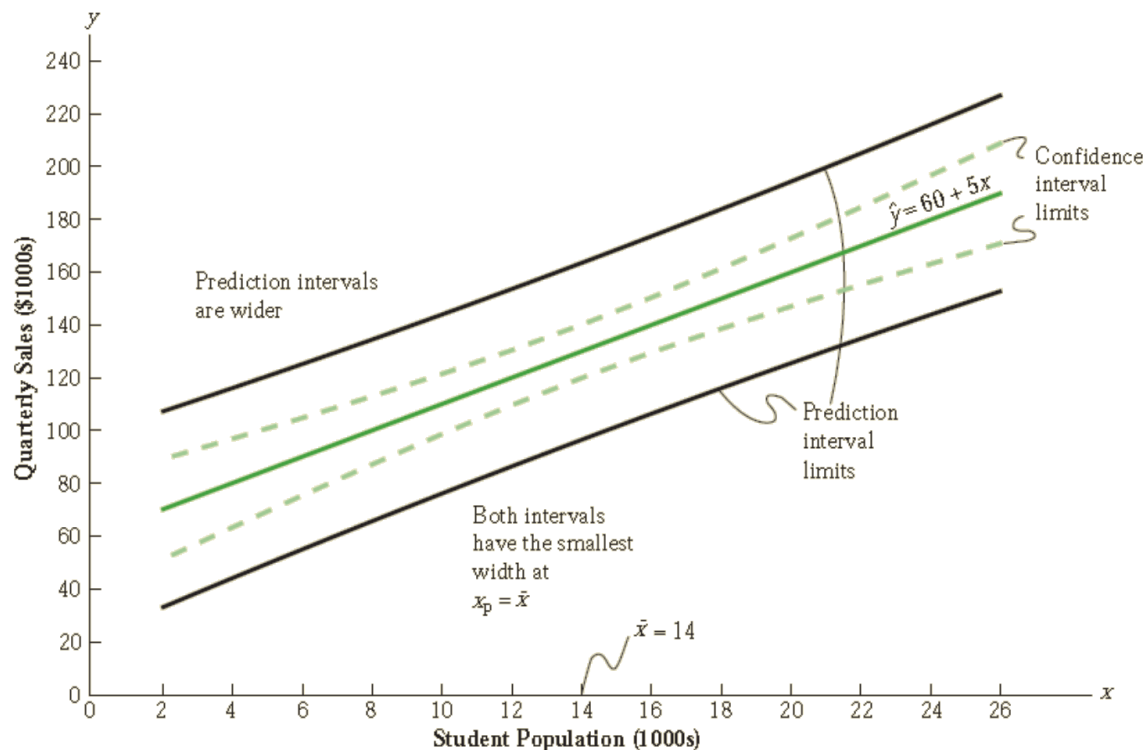
Prediction Interval for an Individual Value of y

$$\hat{y}_p \pm t_{\alpha/2} s_{\text{ind}}$$

$$t_{\alpha/2} s_{\text{ind}} = 2.306(14.69) = 33.875,$$

$$110 \pm 33.875$$

Prediction Interval for an Individual Value of y



Confidence intervals vs prediction intervals

- Confidence intervals and prediction intervals show the precision of the regression results.
- Narrower intervals provide a higher degree of precision

Python Code for Prediction Interval

```
In [43]: from statsmodels.stats.outliers_influence import summary_table  
  
st, data1, ss2 = summary_table(result1, alpha=0.05)  
fittedvalues = data1[:,2]  
predict_mean_se = data1[:,3]  
predict_mean_ci_low, predict_mean_ci_upp = data1[:,4:6].T  
predict_ci_low, predict_ci_upp = data1[:,6:8].T
```

Python Code

```
In [44]: predict_mean_ci_low
```

```
Out[44]: array([ 51.03868339, 75.2931351 , 87.10977127, 87.10977127,  
                109.56629808, 129.56629808, 147.10977127, 147.10977127,  
                155.2931351 , 171.03868339])
```

```
In [45]: predict_mean_ci_upp
```

```
Out[45]: array([ 88.96131661, 104.7068649 , 112.89022873, 112.89022873,  
                130.43370192, 150.43370192, 172.89022873, 172.89022873,  
                184.7068649 , 208.96131661])
```

```
In [46]: predict_ci_low
```

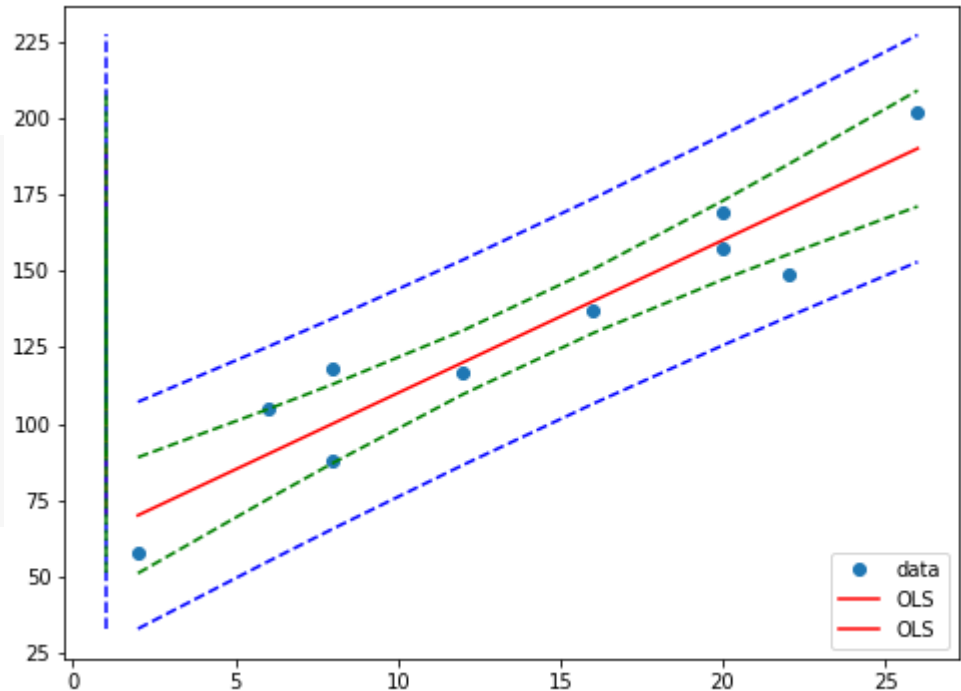
```
Out[46]: array([ 32.89834155, 54.8817226 , 65.60291394, 65.60291394,  
                86.446108 , 106.446108 , 125.60291394, 125.60291394,  
                134.8817226 , 152.89834155])
```

```
In [47]: predict_ci_upp
```

```
Out[47]: array([107.10165845, 125.1182774 , 134.39708606, 134.39708606,  
                153.553892 , 173.553892 , 194.39708606, 194.39708606,  
                205.1182774 , 227.10165845])
```

Python Code

```
In [48]: X = s.add_constant(x)
fig, ax = plt.subplots(figsize=(8,6))
ax.plot(x, y, 'o', label="data")
ax.plot(X, fittedvalues, 'r-', label='OLS')
ax.plot(X, predict_ci_low, 'b--')
ax.plot(X, predict_ci_upp, 'b--')
ax.plot(X, predict_mean_ci_low, 'g--')
ax.plot(X, predict_mean_ci_upp, 'g--')
ax.legend(loc='best');
plt.show()
```



Thank You





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Estimation, Prediction of Regression Model Residual Analysis: Validating Model Assumptions - II

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Understanding different types of residual analysis
- Plotting residual plots using python

Residual Analysis: Validating Model Assumptions

- **Residual analysis** *is the primary tool for determining whether the assumed regression model is appropriate*

RESIDUAL FOR OBSERVATION i

$$y_i - \hat{y}_i$$

where

y_i is the observed value of the dependent variable

$\hat{y}_i$ is the estimated value of the dependent variable

Assumptions about the error term ϵ

$$y = \beta_0 + \beta_1 x + \epsilon$$

1. $E(\epsilon) = 0$.
2. The variance of ϵ , denoted by σ^2 , is the same for all values of x .
3. The values of ϵ are independent.
4. The error term ϵ has a normal distribution.

Importance of the Assumptions

- These assumptions provide the theoretical basis for the t test and the F test used to determine whether the relationship between x and y is significant, and for the confidence and prediction interval estimates
- If the assumptions about the error term ξ appear questionable, the hypothesis tests about the significance of the regression relationship and the interval estimation results may not be valid.

Residuals for Ice cream parlours

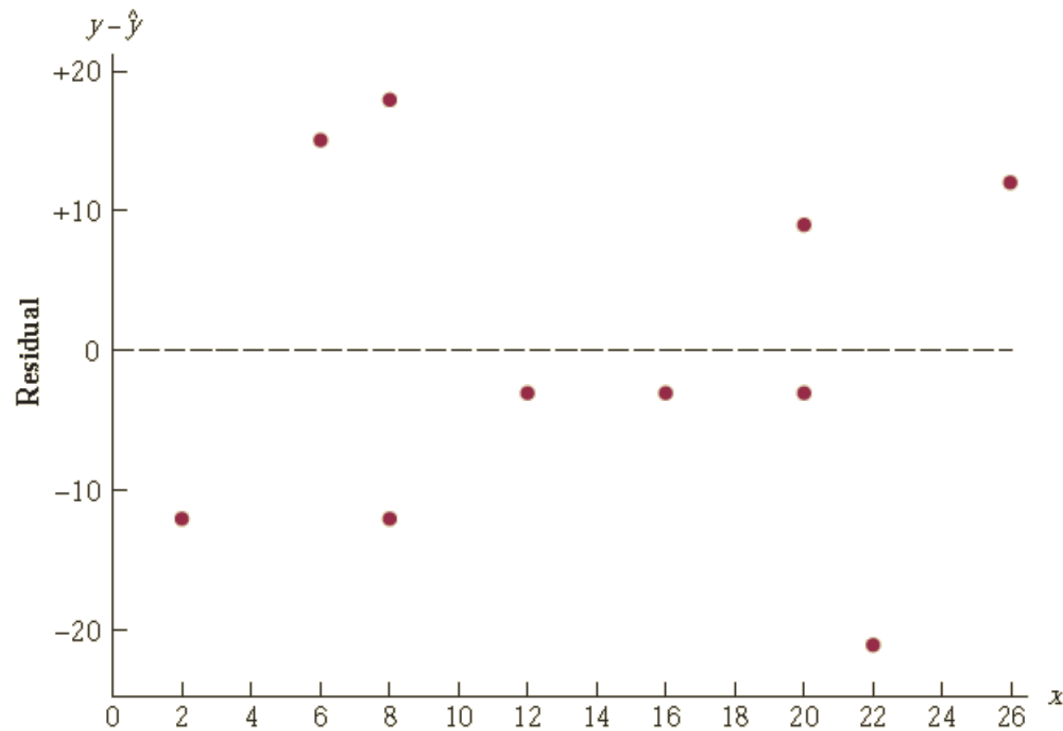
Student Population	Sales	Estimated Sales	Residuals
x_i	y_i	$\hat{y}_i = 60 + 5x_i$	$y_i - \hat{y}_i$
2	58	70	-12
6	105	90	15
8	88	100	-12
8	118	100	18
12	117	120	-3
16	137	140	-3
20	157	160	-3
20	169	160	9
22	149	170	-21
26	202	190	12

Source: Statistics for Business & Economics, David R. Anderson, Dennis J. Sweeney, Thomas A. Williams, Jeffrey D. Camm, James J. Cochran, Cengage Learning, 2013

Residual analysis is based on an examination of graphical plots

- A plot of the residuals against values of the independent variable x
- A plot of residuals against the predicted values of the dependent variable $\hat{y}$
- A standardized residual plot
- A normal probability plot

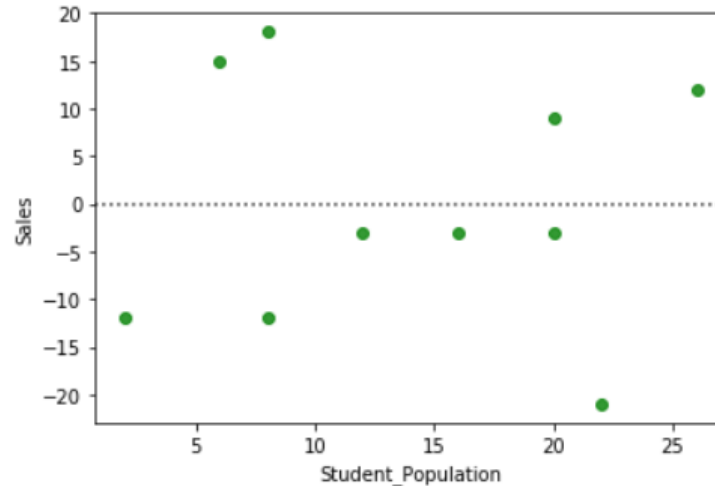
Residual Plot Against x



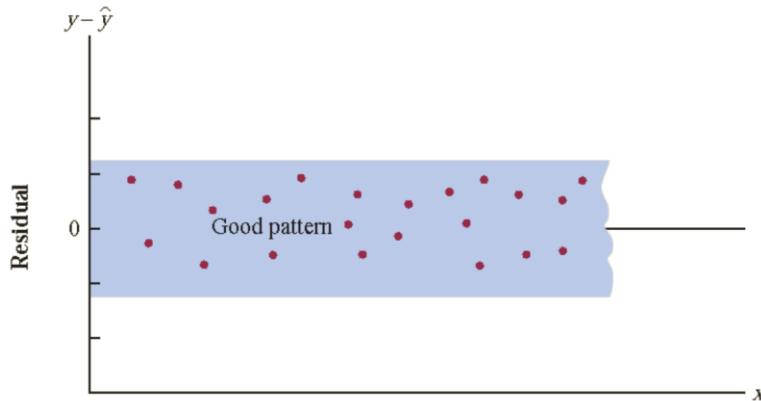
Residual Plot Against x

```
In [18]: import seaborn as sns  
sns.residplot(df1['Student_Population'],df1['Sales'], color="g")
```

```
Out[18]: <matplotlib.axes._subplots.AxesSubplot at 0x24e594e9f60>
```



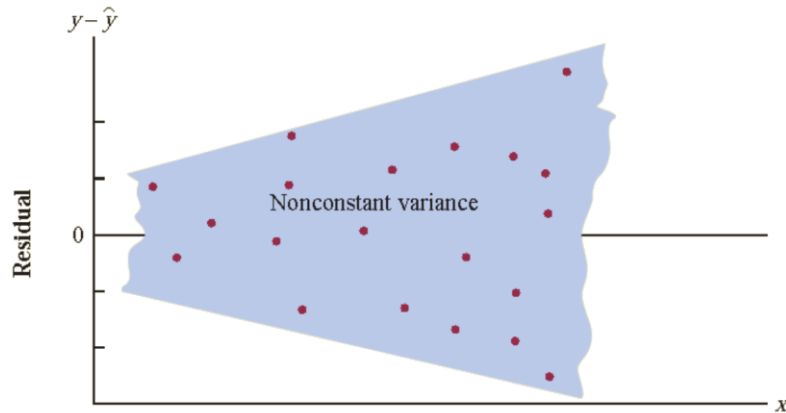
Assumption: the variance is the same for all values of x



- The residual plot should give an overall impression of a horizontal band of points

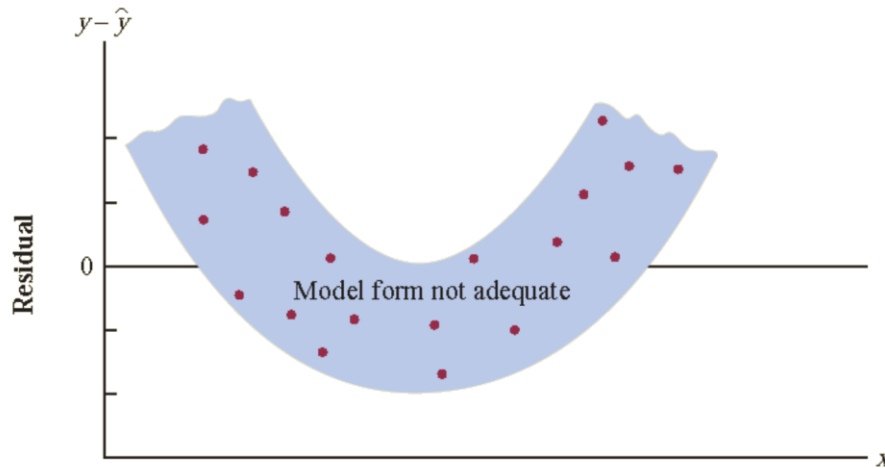
Violation of Assumption:

The variance of 'e' is not the same for all values of x



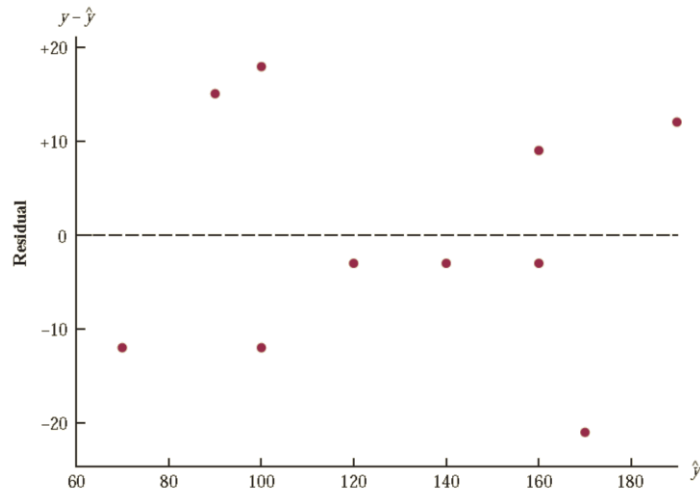
- Assumption of a constant variance of 'e' is violated
- If variability about the regression line is greater for larger values of x

Assumed regression model is not an adequate representation



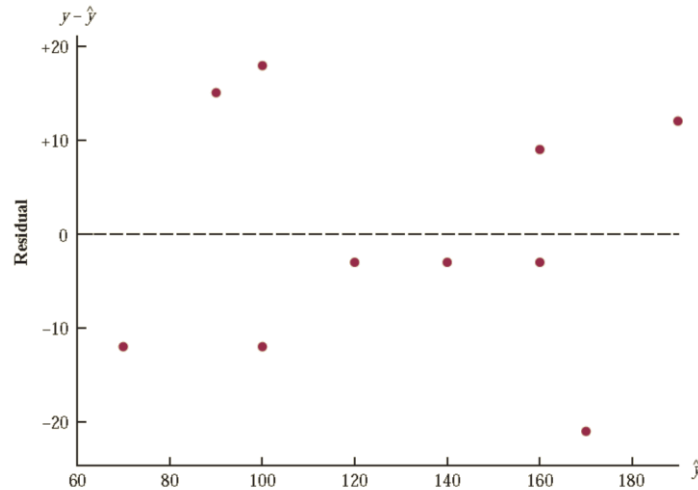
A curvilinear regression model or multiple regression model should be considered.

Residual Plot Against $\hat{y}$



- The pattern of this residual plot is the same as the pattern of the residual plot against the independent variable x .
- It is not a pattern that would lead us to question the model assumptions.

Residual Plot Against $\hat{y}$



- For simple linear regression, both the residual plot against x and the residual plot against $\hat{y}$ provide the same pattern
- For multiple regression analysis, the residual plot against $\hat{y}$ is more widely used because of the presence of more than one independent variable.

Standardized Residuals

- Many of the residual plots provided by computer software packages use a standardized version of the residuals.
- A random variable is standardized by subtracting its mean and dividing the result by its standard deviation.
- With the least squares method, the mean of the residuals is zero.
- Thus, simply dividing each residual by its standard deviation provides the **standardized residual**

Python Code

```
In [14]: import pandas as pd
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm
import matplotlib.pyplot as plt
```

```
In [9]: df1 = pd.read_excel('Icecream.xlsx')
df1
```

Out[9]:

	Student_Population	Sales
0	2	58
1	6	105
2	8	88
3	8	118
4	12	117
5	16	137
6	20	157
7	20	169
8	22	149

```
In [11]: Reg1 = ols(formula = "Sales ~ Student_Population", data = df1)
Fit1 = Reg1.fit()
print(Fit1.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          0.903
Model:                  OLS    Adj. R-squared:       0.891
Method:                 Least Squares    F-statistic:       74.25
Date:                   Thu, 05 Sep 2019    Prob (F-statistic): 2.55e-05
Time:                   11:16:42    Log-Likelihood:    -39.342
No. Observations:      10    AIC:              82.68
Df Residuals:          8    BIC:              83.29
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	60.0000	9.226	6.503	0.000	38.725	81.275
x1	5.0000	0.580	8.617	0.000	3.662	6.338

```

=====
Omnibus:                0.928    Durbin-Watson:          3.224
Prob(Omnibus):          0.629    Jarque-Bera (JB):       0.616
Skew:                   -0.060    Prob(JB):               0.735
Kurtosis:               1.790    Cond. No.                33.6
=====

```


Python Code

```
In [12]: print(anova_lm(Fit1))
```

	df	sum_sq	mean_sq	F	PR(>F)
Student_Population	1.0	14200.0	14200.00	74.248366	0.000025
Residual	8.0	1530.0	191.25	NaN	NaN

Standardized Residuals

STANDARD DEVIATION OF THE i th RESIDUAL

$$s_{y_i - \hat{y}_i} = s \sqrt{1 - h_i}$$

where

$s_{y_i - \hat{y}_i}$ = the standard deviation of residual i

s = the standard error of the estimate

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum (x_i - \bar{x})^2}$$

Computation of standardized residuals for Icecream parlors

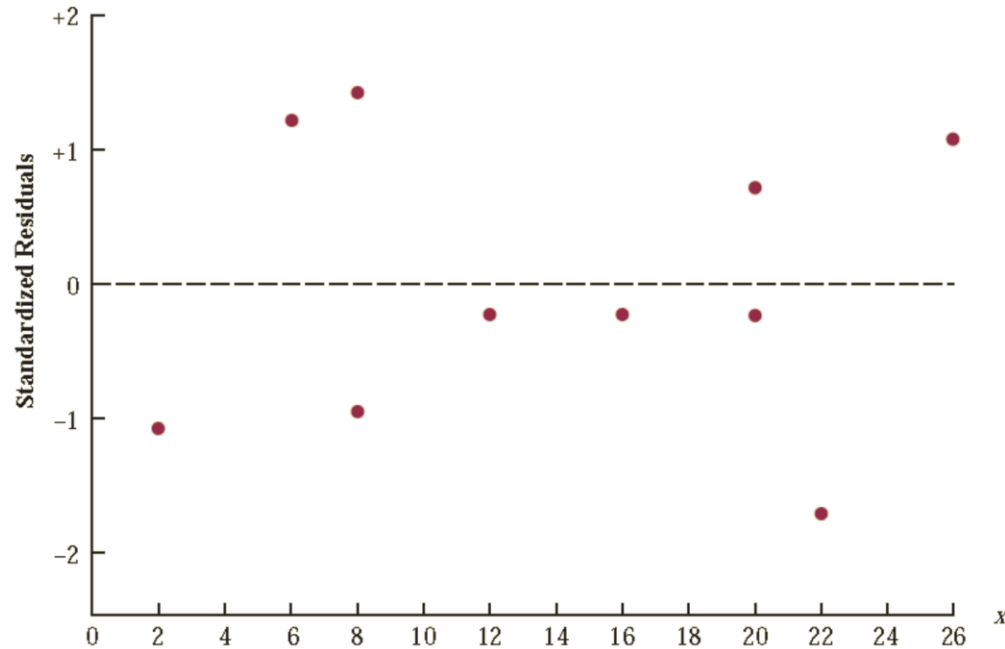
Restaurant i	x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$	$\frac{(x_i - \bar{x})^2}{\sum (x_i - \bar{x})^2}$	h_i	$s_{y_i - \hat{y}_i}$	$y_i - \hat{y}_i$	Standardized Residual
1	2	-12	144	.2535	.3535	11.1193	-12	-1.0792
2	6	-8	64	.1127	.2127	12.2709	15	1.2224
3	8	-6	36	.0634	.1634	12.6493	-12	-.9487
4	8	-6	36	.0634	.1634	12.6493	18	1.4230
5	12	-2	4	.0070	.1070	13.0682	-3	-.2296
6	16	2	4	.0070	.1070	13.0682	-3	-.2296
7	20	6	36	.0634	.1634	12.6493	-3	-.2372
8	20	6	36	.0634	.1634	12.6493	9	.7115
9	22	8	64	.1127	.2127	12.2709	-21	-1.7114
10	26	12	144	.2535	.3535	11.1193	12	1.0792
Total			568					

Computation of standardized residuals for Icecream parlors

STANDARDIZED RESIDUAL FOR OBSERVATION i

$$\frac{y_i - \hat{y}_i}{s_{y_i - \hat{y}_i}}$$

Plot of The Standardized Residuals Against The Independent Variable x



Plot of The Standardized Residuals Against The Independent Variable x

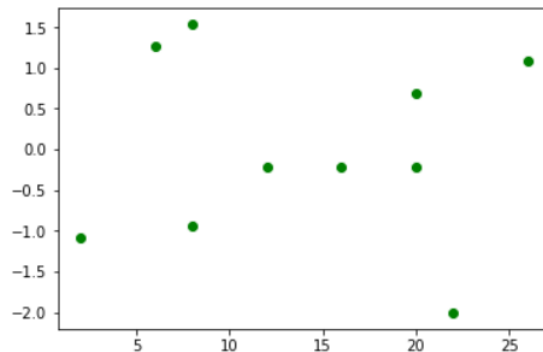
```
In [21]: influence = Fit1.get_influence()  
resid_student = influence.resid_studentized_external
```

```
In [22]: resid_student
```

```
Out[22]: array([-1.09212653,  1.26798654, -0.94196706,  1.54023214, -0.21544891,  
               -0.21544891, -0.22263461,  0.68766487, -2.01063738,  1.09212653])
```

```
In [24]: plt.figure()  
plt.scatter(df1['Student_Population'], resid_student, color = "green")
```

```
Out[24]: <matplotlib.collections.PathCollection at 0x24e5a382b38>
```



Studentized residual

- The standardized residual plot can provide insight about the assumption that the error 'e' term has a normal distribution.
- If this assumption is satisfied, the distribution of the standardized residuals should appear to come from a standard normal probability distribution.

Studentized residual

- Thus, when looking at a standardized residual plot, we should expect to see approximately 95% of the standardized residuals between -2 and 2.
- We see in Figure that for the Armand's example all standardized residuals are between -2 and 2.
- Therefore, on the basis of the standardized residuals, this plot gives us no reason to question the assumption that 'e' has a normal distribution.

Normal Probability Plot

- Another approach for determining the validity of the assumption that the error term has a normal distribution is the **normal probability plot**.
- To show how a normal probability plot is developed, we introduce the concept of *normal scores*.

Normal Probability Plot

- Suppose 10 values are selected randomly from a normal probability distribution with a mean of zero and a standard deviation of one, and that the sampling process is repeated over and over with the values in each sample of 10 ordered from smallest to largest.
- For now, let us consider only the smallest value in each sample.
- The random variable representing the smallest value obtained in repeated sampling is called the first-order statistic.

Normal Probability Plot

NORMAL SCORES
FOR $n = 10$

Order Statistic	Normal Score
1	-1.55
2	-1.00
3	-.65
4	-.37
5	-.12
6	.12
7	.37
8	.65
9	1.00
10	1.55

Normal Probability Plot

Restaurant	$\frac{(x_i - \bar{x})^2}{\sum (x_i - \bar{x})^2}$							Standardized
i	x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$		h_i	$s_{y_i - \hat{y}_i}$	$y_i - \hat{y}_i$	Residual
1	2	-12	144	.2535	.3535	11.1193	-12	-1.0792
2	6	-8	64	.1127	.2127	12.2709	15	1.2224
3	8	-6	36	.0634	.1634	12.6493	-12	-.9487
4	8	-6	36	.0634	.1634	12.6493	18	1.4230
5	12	-2	4	.0070	.1070	13.0682	-3	-.2296
6	16	2	4	.0070	.1070	13.0682	-3	-.2296
7	20	6	36	.0634	.1634	12.6493	-3	-.2372
8	20	6	36	.0634	.1634	12.6493	9	.7115
9	22	8	64	.1127	.2127	12.2709	-21	-1.7114
10	26	12	144	.2535	.3535	11.1193	12	1.0792
Total			568					

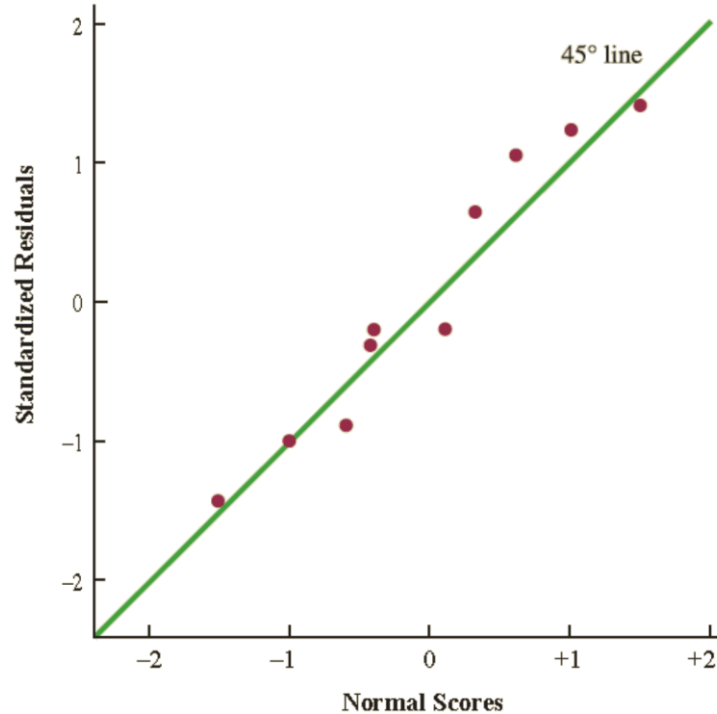
Normal scores and ordered standardized residuals for Armand's pizza parlors

Normal Scores	Ordered Standardized Residuals
-1.55	-1.7114
-1.00	-1.0792
-.65	-.9487
-.37	-.2372
-.12	-.2296
.12	-.2296
.37	.7115
.65	1.0792
1.00	1.2224
1.55	1.4230

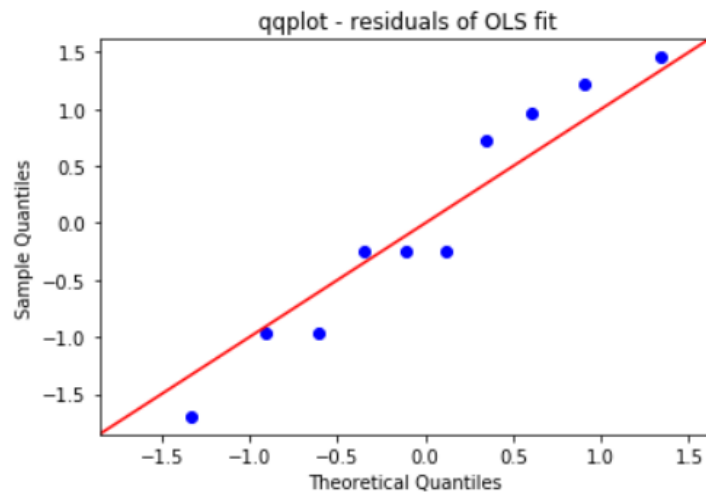
Normal Probability Plot

- If the normality assumption is satisfied, the smallest standardized residual should be close to the smallest normal score, the next smallest standardized residual should be close to the next smallest normal score, and so on.
- If we were to develop a plot with the normal scores on the horizontal axis and the corresponding standardized residuals on the vertical axis, the plotted points should cluster closely around a 45-degree line passing through the origin if the standardized residuals are approximately normally distributed.
- Such a plot is referred to as a *normal probability plot*.

Normal probability plot for Ice Cream parlors



```
In [29]: from scipy import stats
import statsmodels.api as sm
res = Fit1.resid # residuals
probplot = sm.ProbPlot(res,stats.norm, fit=True)
fig = probplot.qqplot(line='45')
h = plt.title(' qqplot - residuals of OLS fit')
plt.show()
```



Thank You





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

MULTIPLE REGRESSION MODEL - I

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Multiple regression model
- Least squares method
- Multiple coefficient of determination
- Model assumptions
- Testing for significance F-Test, t-Test



Multiple regression model

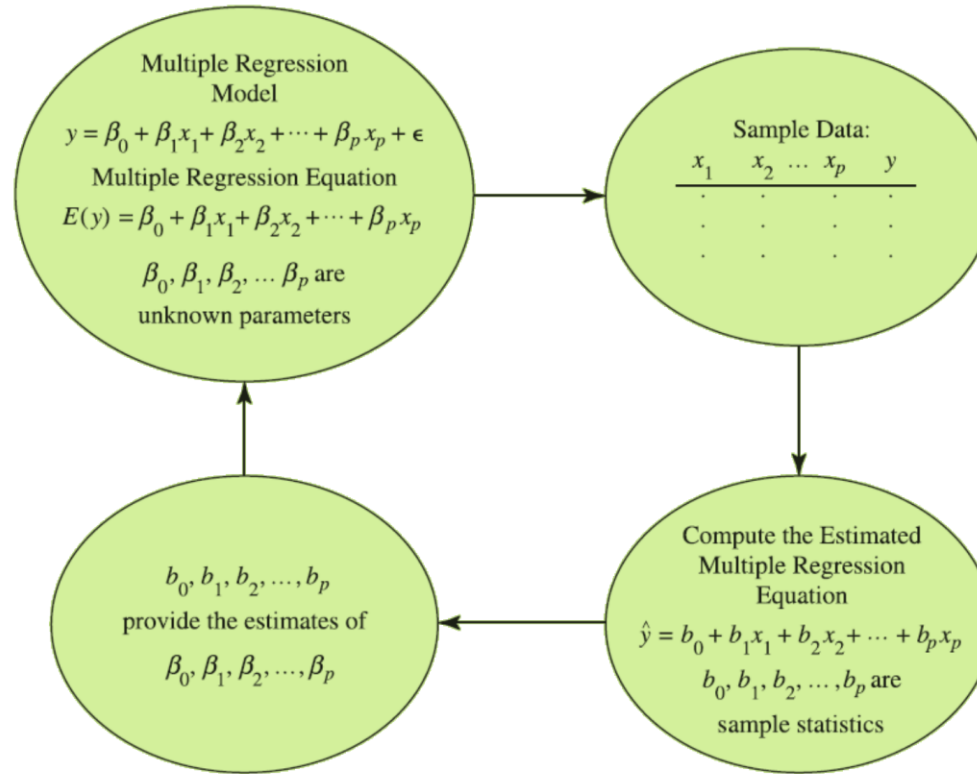
MULTIPLE REGRESSION MODEL

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

MULTIPLE REGRESSION EQUATION

$$E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The estimation process For multiple regression



Simple vs multiple regression

- In simple linear regression, b_0 and b_1 were the sample statistics used to estimate the parameters β_0 and β_1 .
- Multiple regression parallels this statistical inference process, with $b_0, b_1, b_2, \dots, b_p$ denoting the sample statistics used to estimate the parameters $\beta_0, \beta_1, \beta_2, \dots, \beta_p$.

$$b_0 \rightarrow \beta_0$$

$$b_1 \rightarrow \beta_1$$

$$b_2 \rightarrow \beta_2$$

Least Squares Method

LEAST SQUARES CRITERION

$$\min \sum (y_i - \hat{y}_i)^2$$

$$b_0 \quad b_1 \quad \hat{y}_i = b_0 + b_1 x_1$$
$$\hat{y}_i = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_p x_p$$

Least Squares Method

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \cdots + b_px_p$$

ϕ_1

An Example: Trucking Company

- As an illustration of multiple regression analysis, we will consider a problem faced by the Trucking Company.
- A major portion of business involves deliveries throughout its local area.
- To develop better work schedules, the managers want to estimate the total daily travel time for their drivers.

Source: Statistics for Business and Economics, 2012, Anderson

PRELIMINARY DATA FOR BUTLER TRUCKING

Driving Assignment	x_1 = Miles Traveled	y = Travel Time (hours)
1	100	9.3
2	50	4.8
3	100	8.9
4	100	6.5
5	50	4.2
6	80	6.2
7	75	7.4
8	65	6.0
9	90	7.6
10	90	6.1

Using python import data

```
In [1]: import pandas as pd
        from statsmodels.formula.api import ols
        from statsmodels.stats.anova import anova_lm
        import matplotlib.pyplot as plt
```

```
In [2]: df1 = pd.read_excel('Trucking.xlsx')
        df1
```

Using python import data

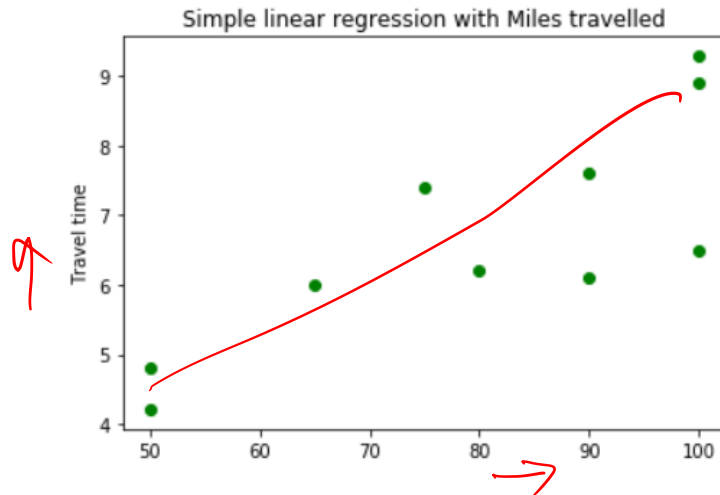
Out[2]:

	Driving Assignmnet	x1	n_of_deliveries	travel_time ^y
0	1	100	4	9.3
1	2	50	3	4.8
2	3	100	4	8.9
3	4	100	2	6.5
4	5	50	2	4.2
5	6	80	2	6.2
6	7	75	3	7.4
7	8	65	4	6.0
8	9	90	3	7.6
9	10	90	2	6.1

Scatter Diagram Of Preliminary Data For Trucking x_1

```
In [3]: import matplotlib.pyplot as plt  
plt.scatter(df1['x1'],df1['travel_time'], color = "green")  
plt.ylabel('Travel time')  
plt.title(' Simple linear regression with Miles travelled ')
```

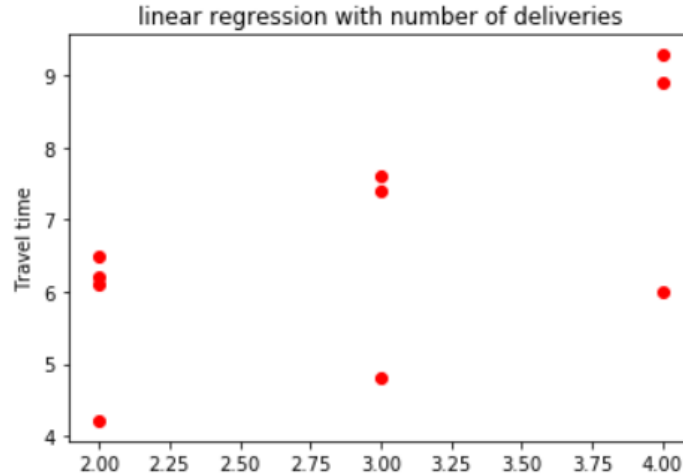
Out[3]: Text(0.5,1,' Simple linear regression with Miles travelled ')



Scatter Diagram Of Preliminary Data For Trucking x_2

```
In [11]: plt.scatter(df1['n_of_deliveries'], df1['travel_time'], color = "red")  
plt.ylabel('Travel time') |  
plt.title('linear regression with number of deliveries')
```

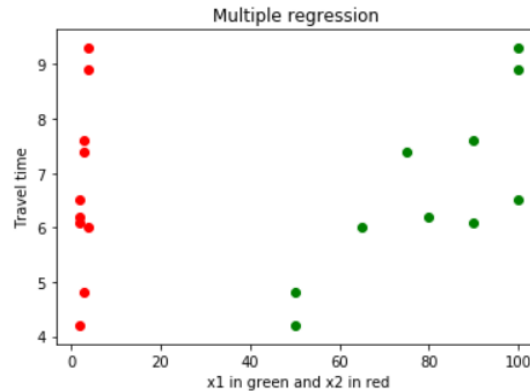
```
Out[11]: Text(0.5,1,'linear regression with number of deliveries')
```



Scatter Diagram For x_1 and x_2

```
In [14]: import matplotlib.pyplot as plt
plt.figure()
plt.scatter(df1['x1'],df1['travel_time'], color = "green")
plt.scatter(df1['n_of_deliveries'], df1['travel_time'], color = "red")
plt.ylabel('Travel time')
plt.title('Multiple regression ') |
plt.xlabel('x1 in green and x2 in red')
```

Out[14]: Text(0.5,0,'x1 in green and x2 in red')



Linear regression Vs. multiple regression model

- Linear regression

$$\hat{y} = 1.27 + .0678x_1$$

Linear regression Vs. multiple regression model

```
In [8]: Reg1 = ols(formula = "travel_time ~ x1", data = df1)
Fit1 = Reg1.fit()
print(Fit1.summary())
```

OLS Regression Results

Dep. Variable:	travel_time	R-squared:	0.664
Model:	OLS	Adj. R-squared:	0.622
Method:	Least Squares	F-statistic:	15.81
Date:	Fri, 06 Sep 2019	Prob (F-statistic):	0.00408
Time:	11:09:17	Log-Likelihood:	-13.092
No. Observations:	10	AIC:	30.18
Df Residuals:	8	BIC:	30.79
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.2739	1.401	0.909	0.390	-1.956	4.504
x1	0.0678	0.017	3.977	0.004	0.028	0.107

Omnibus:	0.694	Durbin-Watson:	1.723
Prob(Omnibus):	0.707	Jarque-Bera (JB):	0.623
Skew:	-0.333	Prob(JB):	0.732
Kurtosis:	1.974	Cond. No.	363.

$$\hat{y} = 1.2739 + 0.0678x_1$$

Linear regression Vs. Multiple regression model

- Multiple regression

$$\hat{y} = -.869 + .0611x_1 + .923x_2$$

Linear regression Vs. Multiple regression model

```
In [15]: from statsmodels.formula.api import ols
model = ols('travel_time ~ x1+n_of_deliveries ', data=df1).fit()
model.summary()

C:\Users\HP\Anaconda3\lib\site-packages\scipy\stats\stats.py:1390: UserWarning:
  g anyway, n=10
  "anyway, n=%i" % int(n))
```

Out[15]: OLS Regression Results

Dep. Variable:	travel_time	R-squared:	0.904			
Model:	OLS	Adj. R-squared:	0.876			
Method:	Least Squares	F-statistic:	32.88			
Date:	Fri, 06 Sep 2019	Prob (F-statistic):	0.000276			
Time:	11:16:53	Log-Likelihood:	-6.8398			
No. Observations:	10	AIC:	19.68			
Df Residuals:	7	BIC:	20.59			
Df Model:	2					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.8687	0.952	-0.913	0.392	-3.119	1.381
x1	0.0611	0.010	6.182	0.000	0.038	0.085
n_of_deliveries	0.9234	0.221	4.176	0.004	0.401	1.446
Omnibus:	0.039	Durbin-Watson:	2.515			
Prob(Omnibus):	0.981	Jarque-Bera (JB):	0.151			
Skew:	0.074	Prob(JB):	0.927			
Kurtosis:	2.418	Cond. No.	435.			

1/0: $\beta_1 = \beta_2 = 0$

$$y = -0.8687 + 0.0611x_1 + 0.9234x_2$$

$x_2 = \text{no. of deliveries}$

Multiple Coefficient of Determination

RELATIONSHIP AMONG SST, SSR, AND SSE

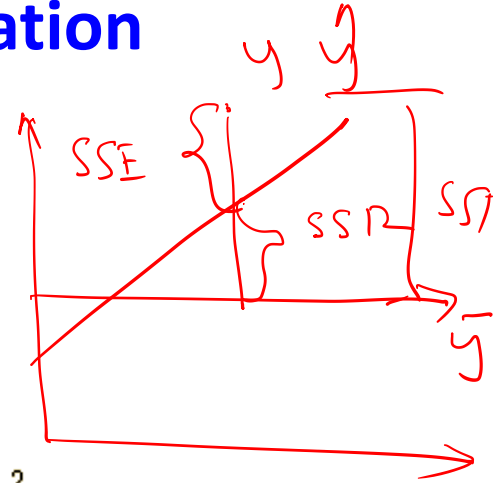
$$SST = SSR + SSE$$

where

$SST = \text{total sum of squares} = \sum (y_i - \bar{y})^2$

$SSR = \text{sum of squares due to regression} = \sum (\hat{y}_i - \bar{y})^2$

$SSE = \text{sum of squares due to error} = \sum (y_i - \hat{y}_i)^2$



Multiple Coefficient of Determination for linear model

In [9]: `print(anova_lm(Fit1))`

	df	sum_sq	mean_sq	F	PR(>F)
x1	1.0	15.871304	15.871304	15.814578	0.00408
Residual	8.0	8.028696	1.003587	NaN	NaN

$$SST = 15.87 + 8.02 = 23.89$$

$$SS_E = 8.02$$

$$SS_R = 15.871$$

Multiple Coefficient of Determination for Multiple regression model

```
In [18]: anova_table = anova_lm(model, typ=1)
         anova_table
```

Out[18]:

	df	sum_sq	mean_sq	F	PR(>F)
x1	1.0	15.871304	15.871304	48.315660	0.000221
n_of_deliveries	1.0	5.729252	5.729252	17.441075	0.004157
Residual	7.0	2.299443	0.328492	NaN	NaN

$$\begin{aligned} SST &= 22 & SSR &= 20.1 \\ SSE &= 2.29 \end{aligned}$$

Multiple Coefficient of Determination

MULTIPLE COEFFICIENT OF DETERMINATION

$$R^2 = \frac{SSR}{SST}$$

$$R^2 = \frac{21.601}{23.900} = .904$$

Multiple Coefficient of Determination

- Adding independent variables causes the prediction errors to become smaller, thus reducing the sum of squares due to error, SSE.
- Because $SSR = SST - SSE$, when SSE becomes smaller, SSR becomes larger, causing $R^2 = SSR/SST$ to increase.
- Many analysts prefer adjusting R^2 for the number of independent variables to avoid overestimating the impact of adding an independent variable on the amount of variability explained by the estimated regression equation.

$$SST = SSR + SSE$$

↓

Adjusted Multiple Coefficient of Determination

n = number of observations

p = denoting the number of independent variables

$$R^2 = \frac{SSR}{SST} = \frac{SST - SSE}{SST}$$

$$= 1 - \frac{SSE}{SST}$$

ADJUSTED MULTIPLE COEFFICIENT OF DETERMINATION

$$R_a^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

$$= 1 - \frac{SSE / (n - p - 1)}{SST / (n - 1)}$$

$$R_a^2 = 1 - (1 - .904) \frac{10 - 1}{10 - 2 - 1} = .88$$

=

OLS Summary

```
In [15]: from statsmodels.formula.api import ols
model = ols('travel_time ~ x1+n_of_deliveries ', data=df1).fit()
model.summary()
```

C:\Users\HP\Anaconda3\lib\site-packages\scipy\stats\stats.py:1390: UserWarning: anyway, n=10
"anyway, n=%i" % int(n)

Out[15]: OLS Regression Results

Dep. Variable:	travel_time	R-squared:	0.904			
Model:	OLS	Adj. R-squared:	0.876			
Method:	Least Squares	F-statistic:	32.88			
Date:	Fri, 06 Sep 2019	Prob (F-statistic):	0.000276			
Time:	11:16:53	Log-Likelihood:	-6.8398			
No. Observations:	10	AIC:	19.68			
Df Residuals:	7	BIC:	20.59			
Df Model:	2					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.8687	0.952	-0.913	0.392	-3.119	1.381
x1	0.0611	0.010	6.182	0.000	0.038	0.085
n_of_deliveries	0.9234	0.221	4.176	0.004	0.401	1.446
Omnibus:	0.039	Durbin-Watson:	2.515			
Prob(Omnibus):	0.981	Jarque-Bera (JB):	0.151			
Skew:	0.074	Prob(JB):	0.927			
Kurtosis:	2.418	Cond. No.	435.			

Handwritten notes illustrating the relationship between variables and statistics:

- R^2 and $adj R^2$ are shown with arrows pointing to the corresponding values in the summary table.
- Handwritten equations show the relationship between y and x_1, x_2, x_3 (e.g., $y \rightarrow (x_1, x_2)$, $y \rightarrow x_1$, $y \rightarrow x_1, x_2$, $y \rightarrow x_1, x_2, x_3$).
- Arrows indicate the direction of influence or relationship between variables and statistics.

Adjusted Multiple Coefficient Vs Multiple Coefficient

- If a variable is added to the model, R^2 becomes larger even if the variable added is not statistically significant.
- The adjusted multiple coefficient of determination compensates for the number of independent variables in the model.

Adjusted Multiple Coefficient Vs Multiple Coefficient

- If the value of R^2 is small and the model contains a large number of independent variables, the adjusted coefficient of determination can take a negative value

Model Assumptions

MULTIPLE REGRESSION MODEL

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

$$E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots +$$

Assumption about error term

1. The error term ε is a random variable with mean or expected value of zero;

$$E(\varepsilon) = 0.$$

Implication: For given values of $x_1, x_2, \dots, x_p$ the expected , or average , value of y is given by $E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$

- This equation represents the average of all possible values of y , that might occur for the given value of $x_1, x_2, \dots, x_p$, by $E(y)$.

Assumption about error term

2. The variance of ϵ is denoted by σ^2 and is the same for all values of the independent variables $x_1, x_2, \dots, x_p$.

Implication: The variance of y about the regression line equals σ^2 and is the same for all values of $x_1, x_2, \dots, x_p$.

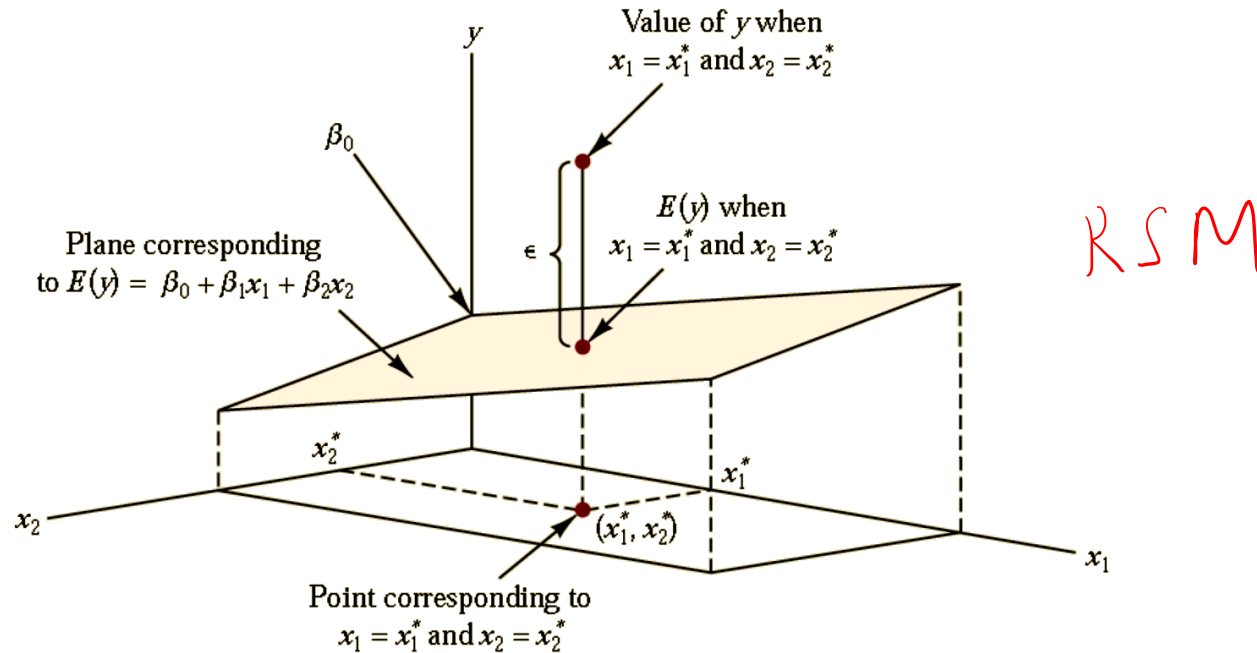
3. The values of ϵ are independent.

Implication: The value of ϵ for a particular set of values for the independent variables is not related to the value of ϵ for any other set of values.

4. The error term ϵ is a normally distributed random variable reflecting the deviation between the y value and the expected value of y given by $\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$.

Implication: Because $\beta_0, \beta_1, \dots, \beta_p$ are constants for the given values of $x_1, x_2, \dots, x_p$, the dependent variable y is also a normally distributed random variable.

Graph of the regression equation for multiple regression analysis with two independent variables



Response variable and response surface

- In regression analysis, the term response variable is often used in place of the term dependent variable.
- Furthermore, since the multiple regression equation generates a plane or surface, its graph is called a response surface.

Thank You





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

MULTIPLE REGRESSION MODEL-II

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Testing for significance
 - F Test
 - t Test
- Python Demo for multiple regression

Testing for Significance

- The F test is used to determine whether a significant relationship exists between the dependent variable and the set of all the independent variables; we will refer to the F test as the test for overall significance.
- If the F test shows an overall significance, the t test is used to determine whether each of the individual independent variables is significant.
- A separate t test is conducted for each of the independent variables in the model; we refer to each of these t tests as a test for individual significance.

F Test

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

The hypotheses for the *F* test involve the parameters of the multiple regression model.

$$H_0: \beta_1 = \beta_2 = \cdots = \beta_p = 0$$

H_a : One or more of the parameters is not equal to zero

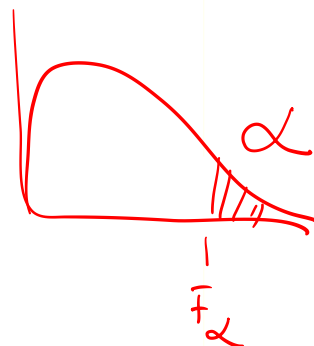
F test significance

$$H_0: \beta_1 = \beta_2 = \cdots = \beta_p = 0$$

H_a : One or more of the parameters is not equal to zero

TEST STATISTIC

$$F = \frac{\text{MSR}}{\text{MSE}} = \frac{\frac{SSR}{p}}{\frac{SSE}{n-p-1}}$$



REJECTION RULE

p -value approach: Reject H_0 if $p\text{-value} \leq \alpha$

Critical value approach: Reject H_0 if $F \geq F_\alpha$

where F_α is based on an F distribution with p degrees of freedom in the numerator and $n - p - 1$ degrees of freedom in the denominator.

F test significance

$$F = \frac{10.8}{.328} = 32.9$$

F Test

```
In [15]: from statsmodels.formula.api import ols
model = ols('travel_time ~ x1+n_of_deliveries ', data=df1).fit()
model.summary()
```

C:\Users\HP\Anaconda3\lib\site-packages\scipy\stats\stats.py:1390: UserWarning
g anyway, n=10
"anyway, n=%i" % int(n))

Out[15]:

OLS Regression Results

Dep. Variable:	travel_time	R-squared:	0.904			
Model:	OLS	Adj. R-squared:	0.976			
Method:	Least Squares	F-statistic:	32.88			
Date:	Fri, 06 Sep 2019	Prob (F-statistic):	0.000276			
Time:	11:16:53	Log-Likelihood:	-6.8398			
No. Observations:	10	AIC:	19.68			
Df Residuals:	7	BIC:	20.59			
Df Model:	2					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.8687	0.952	-0.913	0.392	-3.119	1.381
x1	0.0611	0.010	6.182	0.000	0.038	0.085
n_of_deliveries	0.9234	0.221	4.176	0.004	0.401	1.446
Omnibus:	0.039	Durbin-Watson:	2.515			
Prob(Omnibus):	0.981	Jarque-Bera (JB):	0.151			
Skew:	0.074	Prob(JB):	0.927			
Kurtosis:	2.418	Cond. No.	435.			

ANOVA table

Source	Sum of Squares	Degrees of Freedom	Mean Square	F
Regression	SSR	p	$MSR = \frac{SSR}{p}$	$F = \frac{MSR}{MSE}$
Error	SSE	$n - p - 1$	$MSE = \frac{SSE}{n - p - 1}$	
Total	SST	$n - 1$		

t Test for individual significance

For any parameter β_i

$$H_0: \beta_i = 0$$

$$H_a: \beta_i \neq 0$$

TEST STATISTIC

$$t = \frac{b_i - \beta_i}{s_{b_i}} = \frac{b_i}{s_{b_i}} \quad ($$

REJECTION RULE

p -value approach: Reject H_0 if $p\text{-value} \leq \alpha$

Critical value approach: Reject H_0 if $t \leq -t_{\alpha/2}$ or if $t \geq t_{\alpha/2}$

where $t_{\alpha/2}$ is based on a t distribution with $n - p - 1$ degrees of freedom.

t Test for individual significance

$$b_1 = .061135 \quad s_{b_1} = .009888$$

$$b_2 = .9234 \quad s_{b_2} = .2211$$

$$t = .061135/.009888 = 6.18/$$

$$t = .9234/.2211 = 4.18$$

t Test for individual significance

```
In [15]: from statsmodels.formula.api import ols
model = ols('travel_time ~ x1+n_of_deliveries ', data=df1).fit()
model.summary()

C:\Users\HP\Anaconda3\lib\site-packages\scipy\stats\stats.py:1390: UserWarning:
    g anyway, n=10
    "anyway, n=%i" % int(n))
```

Out[15]:

OLS Regression Results

Dep. Variable:	travel_time	R-squared:	0.904			
Model:	OLS	Adj. R-squared:	0.876			
Method:	Least Squares	F-statistic:	32.88			
Date:	Fri, 06 Sep 2019	Prob (F-statistic):	0.000276			
Time:	11:16:53	Log-Likelihood:	-6.8398			
No. Observations:	10	AIC:	19.68			
Df Residuals:	7	BIC:	20.59			
Df Model:	2					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.8687	0.952	-0.913	0.392	-3.119	1.381
x1	0.0611	0.010	6.182	0.000	0.038	0.085
n_of_deliveries	0.923	0.221	4.176	0.004	0.401	1.446
Omnibus:	0.039	Durbin-Watson:	2.515			
Prob(Omnibus):	0.981	Jarque-Bera (JB):	0.151			
Skew:	0.074	Prob(JB):	0.927			
Kurtosis:	2.418	Cond. No.	435.			

Regression Approach to ANOVA



Regression Approach to ANOVA

- Three different assembly methods, referred to as methods A, B, and C, have been proposed.
- Managers at Chemitech want to determine which assembly method can produce the greatest number of filtration systems per week

Ho: $\mu_A = \mu_B = \mu_C$
Ha: $\mu_A \neq \mu_B \neq \mu_C$

A	B	C
58	58	48
64	69	57
55	71	59
66	64	47
67	68	49

ANOVA

Anova: Single Factor						
SUMMARY						
Groups	Count	Sum	Average	Variance		
A	5	310	62	27.5		
B	5	330	66	26.5		
C	5	260	52	31		
ANOVA						
Source of Variation	SS	df	MS	F	P-value	F crit
Between Groups	520	2	260	9.176471	0.003818	3.885294
Within Groups	340	12	28.33333			
Total	860	14				



Dummy variables for the chemitech experiment

A

1

0

0

B

0

1

0

Observation is associated with assembly method A

Observation is associated with assembly method B

Observation is associated with assembly method C

3-1

Dummy variables for the chemitech experiment

$$\begin{aligned} E(y) &= \text{Expected value of the number of units produced per week} \\ &= \beta_0 + \beta_1 A + \beta_2 B \end{aligned}$$

- If we are interested in the expected value of the number of units assembled per week for an employee who uses method C, our procedure for assigning numerical values to the dummy variables would result in setting $A = B = 0$.
$$E(y) = \beta_0 + \beta_1(0) + \beta_2(0) = \beta_0$$
- The multiple regression equation then reduces to



Dummy variables for the chemitech experiment

- For method A the values of the dummy variables are $A = 1$ and $B = 0$, and

$$E(y) = \beta_0 + \beta_1(1) + \beta_2(0) = \underline{\beta_0 + \beta_1}$$

- For method B we s $E(y) = \beta_0 + \beta_1(0) + \beta_2(1) = \underline{\beta_0} + \underline{\beta_2}$

SUMMARY OUTPUT								
Regression Statistics								
Multiple R	0.777593186							
R Square	0.604651163							
Adjusted R Square	0.53875969							
Standard Error	5.322906474							
Observations	15							
ANOVA								
	df	SS	MS	F	Significance F			
Regression	2	520	260	9.176471	0.003818412			
Residual	12	340	28.33333					
Total	14	860						
	Coefficients	Standard Error	t Stat	P-value	Lower 95%	Upper 95%	Lower 95.0%	Upper 95.0%
Intercept	52	2.380476143	21.84437	4.97E-11	46.81338804	57.18661196	46.81338804	57.18661196
A	10	3.366501646	2.970443	0.011692	2.665023022	17.33497698	2.665023022	17.33497698
B	14	3.366501646	4.15862	0.001326	6.665023022	21.33497698	6.665023022	21.33497698



Estimation of $E(y)$

- $b_0 = 52$
- $b_1 = 10$
- $b_2 = 14$

Assembly Method	Estimation of $E(y)$
A	$b_0 + b_1 = 52 + 10 = \underline{62}$
B	$b_0 + b_2 = 52 + 14 = 66$
C	52

Testing the significance

$$H_0: \beta_1 = \beta_2 = 0$$



Thank You





IIT ROORKEE



NPTEL ONLINE
CERTIFICATION COURSE

Categorical Variable Regression

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES



Agenda

- Purpose of this lecture is to show how categorical variables are handled in regression analysis.
- To illustrate the use and interpretation of a categorical independent variable, we will consider two problems
- Demo on python

What are dummy variables?

- Dummy variables, also called indicator variables allow us to include categorical data (like Gender) in regression models
- A dummy variable can take only 2 values, 0 (absence of a category) and 1 (presence of a category)

Example 1: Problem / Background

- Johnson Filtration, Inc., provides maintenance service for water-filtration systems.
- Customers contact Johnson with requests for maintenance service on their water-filtration systems
- To estimate the service time and the service cost, Johnson's managers want to predict the repair time necessary for each maintenance request
- Hence, repair time in hours is the dependent variable
- Repair time is believed to be related to two factors,
 - the number of months since the last maintenance service
 - the type of repair problem (mechanical or electrical).



Source: Statistics for Business & Economics, David R. Anderson, Dennis J. Sweeney, Thomas A. Williams, Jeffrey D. Camm, James J. Cochran, Cengage Learning, 2013

Data for the Johnson filtration example

service call	months_since_last_service	type_of_repair	repair_time_in_hours
1	2	electrical	2.9
2	6	mechanical	3
3	8	electrical	4.8
4	3	mechanical	1.8
5	2	electrical	2.9
6	7	electrical	4.9
7	9	mechanical	4.2
8	8	mechanical	4.8
9	4	electrical	4.4
10	6	electrical	4.5

```
In [23]: import pandas as pd
import matplotlib as mpl
import statsmodels.formula.api as sm
from sklearn.linear_model import LinearRegression
from scipy import stats
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as s
```

```
In [24]: tbl = pd.read_excel('dummy.xlsx')
tbl
```

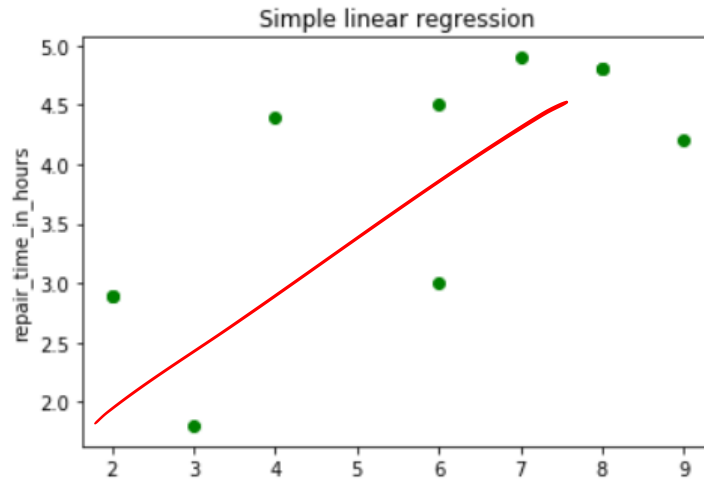
Out[24]:

	servicecall	months_since_last_service	type_of_repair	repair_time_in_hours
0	1	2	electrical	2.9
1	2	6	mechanical	3.0
2	3	8	electrical	4.8
3	4	3	mechanical	1.8
4	5	2	electrical	2.9
5	6	7	electrical	4.9
6	7	9	mechanical	4.2
7	8	8	mechanical	4.8
8	9	4	electrical	4.4
9	10	6	electrical	4.5

Linear Regression

```
In [41]: plt.scatter(tbl['months_since_last_service'], tbl['repair_time_in_hours'], color = "green")  
plt.ylabel('repair_time_in_hours')  
plt.title(' Simple linear regression ')
```

```
Out[41]: Text(0.5,1,' Simple linear regression ')
```



OLS Summary

```
In [44]: from statsmodels.formula.api import ols
Reg = ols(formula = "repair_time_in_hours ~ months_since_last_service", data = tbl)
Fit1 = Reg.fit()
print(Fit1.summary())
```

OLS Regression Results

Dep. Variable:	repair_time_in_hours	R-squared:	0.534
Model:	OLS	Adj. R-squared:	0.476
Method:	Least Squares	F-statistic:	9.174
Date:	Sat, 07 Sep 2019	Prob (F-statistic):	0.0163
Time:	13:26:03	Log-Likelihood:	-10.602
No. Observations:	10	AIC:	25.20
Df Residuals:	8	BIC:	25.81
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	2.1473	0.605	3.549	0.008	0.752	3.542
months_since_last_service	0.3041	0.100	3.029	<u>0.016</u>	0.073	0.536

Omnibus:	0.907	Durbin-Watson:	2.154
Prob(Omnibus):	0.635	Jarque-Bera (JB):	0.751
Skew:	-0.501	Prob(JB):	0.687
Kurtosis:	2.107	Cond. No.	15.1

$$y = 2.1473 + 0.3041 \text{ months}$$

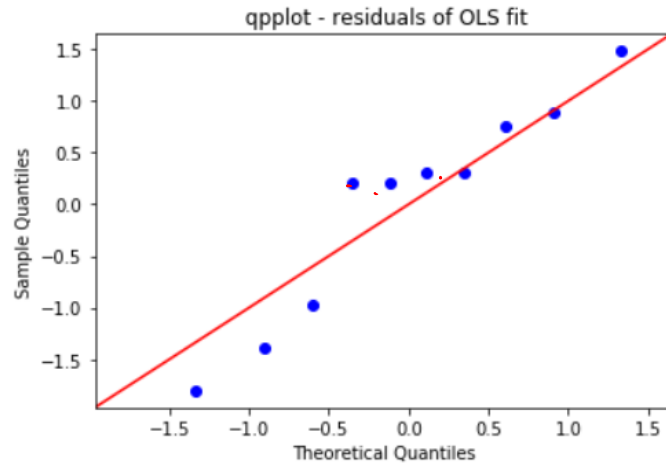
Linear regression

$$y = \beta_0 + \beta_1 x_1 + \epsilon$$

$$\hat{y} = 2.15 + .304x_1$$

Normal probability plot

```
In [49]: res = Fit1.resid # residuals
probplot = s.ProbPlot(res,stats.norm, fit=True)
fig = probplot.qqplot(line='45')
h = plt.title(' qqplot - residuals of OLS fit')
plt.show()
```



Creating dummies

```
In [34]: just_dummies = pd.get_dummies(tbl['type_of_repair'])  
just_dummies
```

Out[34]:

	electrical	mechanical
0	1	0
1	0	1
2	1	0
3	0	1
4	1	0
5	1	0
6	0	1
7	0	1
8	1	0
9	1	0

$$y = a + b_1x_1 + b_2x_2$$

$$y = a + b_1x_1 + b_2(1)$$

$$y = a + b_1x_1 + b_2(0)$$

DATA FOR THE JOHNSON FILTRATION EXAMPLE WITH TYPE OF REPAIR
INDICATED BY A DUMMY VARIABLE ($x_2 = 0$ FOR MECHANICAL; $x_2 = 1$
FOR ELECTRICAL)

Customer	Months Since Last Service (x_1)	Type of Repair (x_2)	Repair Time in Hours (y)
1	2	1	2.9
2	6	0	3.0
3	8	1	4.8
4	3	0	1.8
5	2	1	2.9
6	7	1	4.9
7	9	0	4.2
8	8	0	4.8
9	4	1	4.4
10	6	1	4.5

Adding dummies to table

```
In [38]: just_dummies = pd.get_dummies(tbl['type_of_repair'])
step_1 = pd.concat([tbl, just_dummies], axis=1)
step_1
step_1.drop(['type_of_repair', 'mechanical'], inplace=True, axis=1)

# to run the regression we want to get rid of the strings 'mechanical' and 'electrical'
# and we want to get rid of one dummy variable to avoid the dummy variable trap
# arbitrarily chose "mechanical", coefficients on "electrical" would show effect of "electrical"
# relative to "mechanical"
```

```
In [39]: step_1
```

```
Out[39]:
```



	servicecall	months_since_last_service	repair_time_in_hours	electrical
0	1	2	2.9	1
1	2	6	3.0	0
2	3	8	4.8	1
3	4	3	1.8	0
4	5	2	2.9	1
5	6	7	4.9	1
6	7	9	4.2	0
7	8	8	4.8	0
8	9	4	4.4	1
9	10	6	4.5	1

OLS Summary

```
In [20]: result = sm.OLS(step_1['repair_time_in_hours'], s.add_constant(step_1[['months_since_last_service', 'electrical']])).fit()  
print (result.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	repair_time_in_hours	R-squared:		0.859		
Model:	OLS	Adj. R-squared:		0.819		
Method:	Least Squares	F-statistic:		21.36		
Date:	Sat, 07 Sep 2019	Prob (F-statistic):		0.00105		
Time:	13:08:09	Log-Likelihood:		-4.6200		
No. Observations:	10	AIC:		15.24		
Df Residuals:	7	BIC:		16.15		
Df Model:	2					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	0.9305	0.467	1.993	0.087	-0.174	2.035
months_since_last_service	0.3876	0.063	6.195	0.000	0.240	0.536
electrical	1.2627	0.314	4.020	0.005	0.520	2.005
=====						
Omnibus:	3.357	Durbin-Watson:		1.136		
Prob(Omnibus):	0.187	Jarque-Bera (JB):		1.663		
Skew:	0.994	Prob(JB):		0.435		
Kurtosis:	2.795	Cond. No.		22.0		
=====						

$$y = 0.9305 + 0.3876 \text{ months_since_last_service} + 1.2627 \text{ electrical}$$

Dummy regression

$$x_2 = \begin{cases} 0 & \text{if the type of repair is mechanical} \\ 1 & \text{if the type of repair is electrical} \end{cases}$$

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon$$

$$\hat{y} = .93 + .388x_1 + 1.26x_2$$

$x_2 = 1$ - Electrical

$x_2 = 0$ - Mechanical

Interpreting the Parameters

$$E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$E(y \mid \text{mechanical}) = \beta_0 + \beta_1 x_1 + \beta_2(0) = \beta_0 + \beta_1 x_1 \quad \text{Equation 1}$$

$$\begin{aligned} E(y \mid \text{electrical}) &= \beta_0 + \beta_1 x_1 + \beta_2(1) = \beta_0 + \beta_1 x_1 + \beta_2 \\ &= (\beta_0 + \beta_2) + \beta_1 x_1 \end{aligned} \quad \text{Equation 2}$$

Interpreting the Parameters

- Comparing equations, we see that the mean repair time is a linear function of x_1 for both mechanical and electrical repairs.
- The slope of both equations is β_1 , but the y-intercept differs.
- The y-intercept is β_0 in equation 1 for mechanical repairs and $(\beta_0 + \beta_2)$ in equation 2 for electrical repairs.

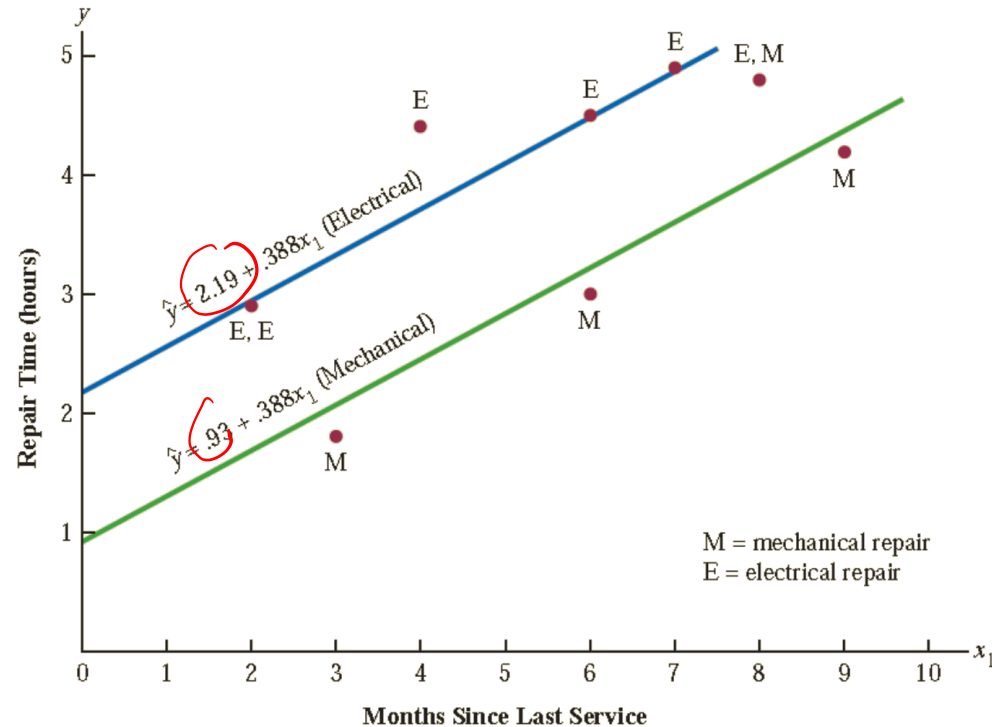
Interpreting the Parameters

- The interpretation of β_2 is that it indicates the difference between the mean repair time for an electrical repair and the mean repair time for a mechanical repair.
- If β_2 is positive, the mean repair time for an electrical repair will be greater than that for a mechanical repair; if β_2 is negative, the mean repair time for an electrical repair will be less than that for a mechanical repair.
- Finally, if $\beta_2 = 0$, there is no difference in the mean repair time between electrical and mechanical repairs and the type of repair is not related to the repair time.

Interpreting the Parameters

- In effect, the use of a dummy variable for type of repair provides two estimated regression equations that can be used to predict the repair time, one corresponding to mechanical repairs and one corresponding to electrical repairs.
- In addition, with $\beta_2 = 1.26$, we learn that, on average, electrical repairs require 1.26 hours longer than mechanical repairs.

Interpreting the Parameters



More Complex Categorical Variables

- A categorical variable with k levels must be modeled using $k - 1$ dummy variables.
- Care must be taken in defining and interpreting the dummy variables.

Example 2: Problem / Background

- The manager of a small sales force wants to know whether average monthly salary is different for males and females in the sales force.
- He obtains data on monthly salary and experience (in months) for each of the 9 employees as shown on the next slide.



Data

Employee	Salary	Gender	Experience
1	7.5	Male	6
2	8.6	Male	10
3	9.1	Male	12
4	10.3	Male	18
5	13	Male	30
6	6.2	Female	5
7	8.7	Female	13
8	9.4	Female	15
9	9.8	Female	21

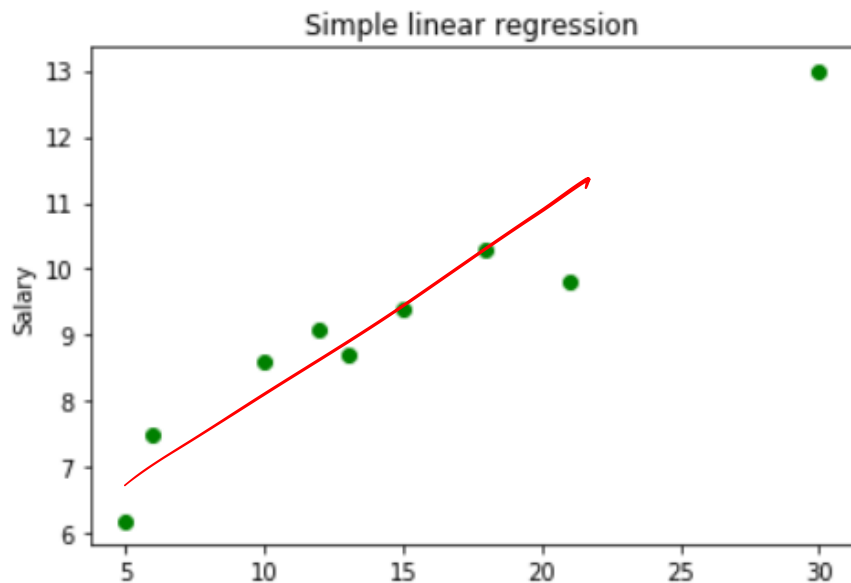
In [50]: `tbl2 = pd.read_excel('dummy2.xlsx')`
`tbl2`

Out[50]:

	Employee	Salary	Gender	Experience
0	1	7.5	Male	6
1	2	8.6	Male	10
2	3	9.1	Male	12
3	4	10.3	Male	18
4	5	13.0	Male	30
5	6	6.2	Female	5
6	7	8.7	Female	13
7	8	9.4	Female	15
8	9	9.8	Female	21

```
In [51]: plt.scatter(tbl2['Experience'], tbl2['Salary'], color = "green")  
plt.ylabel('Salary')  
plt.title(' Simple linear regression ')
```

```
Out[51]: Text(0.5,1,' Simple linear regression ')
```




```
In [59]: Reg2 = ols(formula = "Salary ~ Experience", data = tbl2)
Fit2 = Reg2.fit()
print(Fit2.summary())
```

OLS Regression Results

Dep. Variable:	Salary	R-squared:	0.926			
Model:	OLS	Adj. R-squared:	0.915			
Method:	Least Squares	F-statistic:	87.61			
Date:	Sat, 07 Sep 2019	Prob (F-statistic):	3.30e-05			
Time:	14:18:45	Log-Likelihood:	-6.2491			
No. Observations:	9	AIC:	16.50			
Df Residuals:	7	BIC:	16.89			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

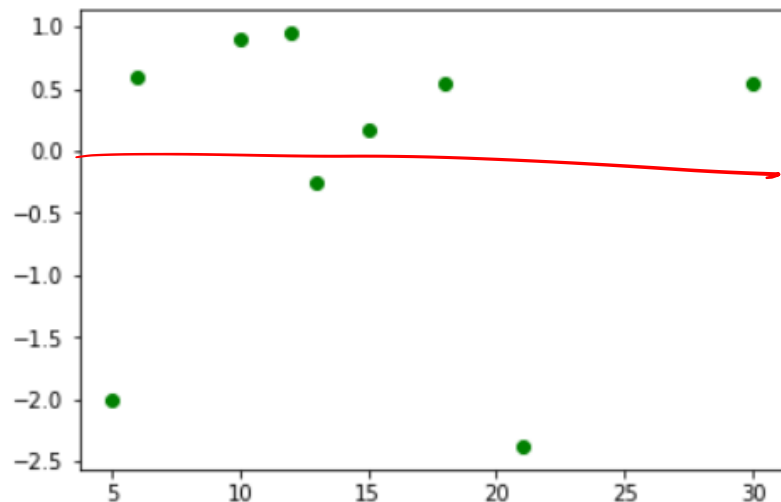
Intercept	5.8093	0.404	14.386	0.000	4.854	6.764
<u>Experience</u>	0.2332	0.025	9.360	0.000	0.174	0.292
=====						
Omnibus:	2.443	Durbin-Watson:	1.171			
Prob(Omnibus):	0.295	Jarque-Bera (JB):	1.432			
Skew:	-0.918	Prob(JB):	0.489			
Kurtosis:	2.331	Cond. No.	35.8			

$$y = 5.84 + 0.2332 \text{ Exp.}$$

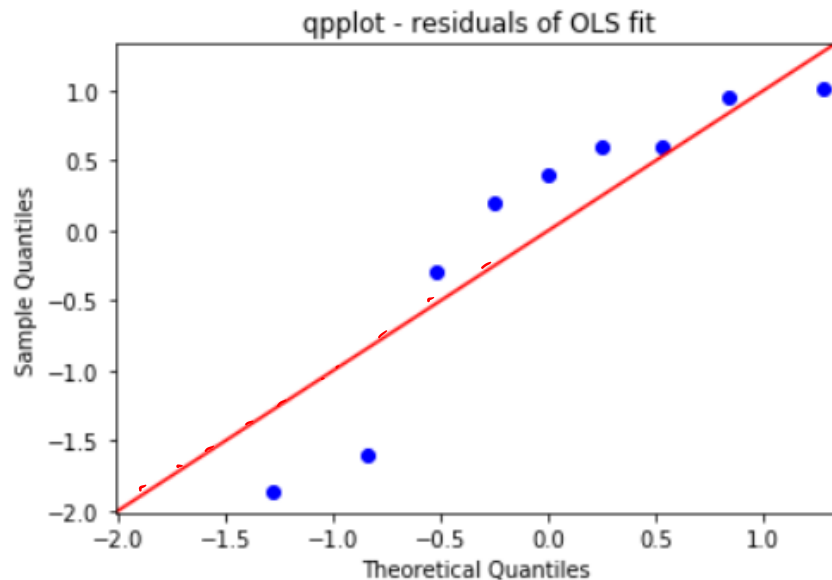
```
In [55]: influence = Fit2.get_influence()  
resid_student = influence.resid_studentized_external
```

```
In [57]: plt.figure()  
plt.scatter(tbl2['Experience'], resid_student, color = "green")
```

```
Out[57]: <matplotlib.collections.PathCollection at 0x2d3e12019b0>
```



```
In [58]: res = Fit2.resid # residuals
probplot = s.ProbPlot(res,stats.norm, fit=True)
fig = probplot.qqplot(line='45')
h = plt.title(' qqplot - residuals of OLS fit')
plt.show()
```



Creating a dummy variable for gender

- Categorical data is included in regression analysis by using dummy variables
- For example, we can assign a value of 0 for males and 1 for females in our data so that a MR model can be developed

$x_i = 0$ males
 $x_i = 1$ females

Employee	Salary	Gender
1	7.5	0
2	8.6	0
3	9.1	0
4	10.3	0
5	13	0
6	6.2	1
7	8.7	1
8	9.4	1
9	9.8	1

```
In [24]: just_dummies2 = pd.get_dummies(tbl2['Gender'])  
just_dummies2
```

Out[24]:

	Female	Male
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
5	1	0
6	1	0
7	1	0
8	1	0

```
In [62]: step_1 = pd.concat([tbl2, just_dummies2], axis=1)
step_1.drop(['Gender', 'Male'], inplace=True, axis=1)
# to run the regression we want to get rid of the strings 'male' and 'female'
# and we want to get rid of one dummy variable to avoid the dummy variable trap
# arbitrarily chose "male", coefficients on "female" would show effect of "female"
# relative to "male"

result = sm.OLS(step_1['Salary'], s.add_constant(step_1[['Female']])).fit()
print (result.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Salary    R-squared:                0.107
Model:                  OLS      Adj. R-squared:           -0.020
Method:                 Least Squares    F-statistic:         0.8426
Date:                  Sat, 07 Sep 2019    Prob (F-statistic):    0.389
Time:                  14:23:57    Log-Likelihood:       -17.455
No. Observations:      9          AIC:                    38.91
Df Residuals:          7          BIC:                    39.30
Df Model:              1
Covariance Type:       nonrobust

=====
                    coef    std err          t      P>|t|      [0.025    0.975]
-----
const              9.7000      0.853     11.367      0.000      7.682     11.718
Female             -1.1750      1.280     -0.918      0.389     -4.202      1.852
=====

Omnibus:                 0.387    Durbin-Watson:           1.912
Prob(Omnibus):           0.824    Jarque-Bera (JB):         0.280
Skew:                    0.330    Prob(JB):                 0.869
Kurtosis:                2.441    Cond. No.                 2.51
=====
```

$$y = 9.7 - 1.1750x_1$$

More on the intercept and slope

- The value of the intercept, 9.70, is the average salary for males (as we coded gender=1 for females and 0 for males)
- The value of the slope, -1.175, tells us that the average females salary is lower than the average male salary by 1.175

```
In [25]: step_1 = pd.concat([tbl2, just_dummies2], axis=1)
step_1.drop(['Gender', 'Male'], inplace=True, axis=1)
# to run the regression we want to get rid of the strings 'male' and 'female'
# and we want to get rid of one dummy variable to avoid the dummy variable trap
# arbitrarily chose "male", coefficients on "female" would show effect of "female"
# relative to "male"

result = sm.OLS(step_1['Salary'], s.add_constant(step_1[['Experience', 'Female']])).fit()
print (result.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Salary      R-squared:                0.974
Model:                  OLS        Adj. R-squared:            0.965
Method:                 Least Squares   F-statistic:             111.6
Date:                  Sat, 07 Sep 2019   Prob (F-statistic):      1.80e-05
Time:                  12:33:40         Log-Likelihood:          -1.5752
No. Observations:      9              AIC:                    9.150
Df Residuals:          6              BIC:                    9.742
Df Model:              2
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	6.2485	0.291	21.439	0.000	5.535	6.962
Experience	0.2271	0.016	14.089	0.000	0.188	0.267
Female	-0.7890	0.238	-3.309	0.016	-1.372	-0.206

```

=====
Omnibus:                0.110   Durbin-Watson:           2.181
Prob(Omnibus):          0.947   Jarque-Bera (JB):        0.198
Skew:                   0.174   Prob(JB):                 0.906
Kurtosis:               2.363   Cond. No.                 44.8
=====

```

$$y = 6.2485 + 0.2271 \text{Exp} - 0.7890 \text{F}$$

What would have happened if we had used 0 for females and 1 for males in our data? Would our results be any different?

```
In [63]: step_1 = pd.concat([tbl2, just_dummies2], axis=1)
step_1.drop(['Gender', 'Female'], inplace=True, axis=1)

result = sm.OLS(step_1['Salary'], s.add_constant(step_1[['Male']])).fit()
print (result.summary())
```

```
OLS Regression Results
=====
Dep. Variable:          Salary    R-squared:                0.107
Model:                  OLS      Adj. R-squared:            -0.020
Method:                 Least Squares    F-statistic:           0.8426
Date:                  Sat, 07 Sep 2019    Prob (F-statistic):      0.389
Time:                  14:27:56    Log-Likelihood:         -17.455
No. Observations:      9          AIC:                   38.91
Df Residuals:          7          BIC:                   39.30
Df Model:              1
Covariance Type:       nonrobust
=====
                    coef    std err          t      P>|t|      [0.025     0.975]
-----
const             8.5250     0.954      8.935     0.000      6.269     10.781
Male             1.1750     1.280      0.918     0.389     -1.852      4.202
=====
Omnibus:            0.387    Durbin-Watson:           1.912
Prob(Omnibus):      0.824    Jarque-Bera (JB):         0.280
Skew:               0.330    Prob(JB):                 0.869
Kurtosis:           2.441    Cond. No.                 2.77
=====
```

Male = 1, female = 0

- Not really – With coding as above, the intercept would change to 8.525 (the average female salary), the slope for gender would still be 1.175, but now it would have a positive sign (reflecting that average male salary is higher than average female salary by 1.175).
Predicted salaries from the model for males / females would not change no matter how dummy variable is coded

More on dummy variables

- For gender, we had only 2 categories – female and male – thus we used a single 0/1 variable for this
- When there are more than 2 categories, the number of dummy variables that should be used equals the number of categories minus 1
- No. of Dummy Variables = No. of levels - 1

Example: Salary vs. Job Grade

- In this example, the categorical variable job grade has 3 levels, 1 (lowest grade), 2, and 3 (highest job grade)

Employee	Job Grade	Salary (\$000)
1	1	7.5
2	3	8.6
3	2	9.1
4	3	10.3
5	3	13
6	1	6.2
7	2	8.7
8	2	9.4
9	3	9.8

Representing 3-level Job Grade using dummy variables Job_1 and Job_2

Employee's Job Grade	Dummy Variables	
	Job_1	Job_2
	1	0
	0	1
3	0	0

Job Grade 3 is the reference category

Data file with dummy variables for job grade

Employee	Job Grade	Salary	Job_1	Job_2
1	1	7.5	1	0
2	3	8.6	0	0
3	2	9.1	0	1
4	3	10.3	0	0
5	3	13	0	0
6	1	6.2	1	0
7	2	8.7	0	1
8	2	9.4	0	1
9	3	9.8	0	0

Thank You

